

LAPORAN MILESTONE 2 TUGAS BESAR IF2224 TEORI BAHASA FORMAL DAN OTOMATA

ARION COMPILER

Syntax Analysis

DEADLINE: Kamis, 7 Mei 2026 Pukul 23:59 WIB



Dipersiapkan oleh:

Kelompok AZZAM | Kode MAR | Kelas 01

13524017	Aziza Dharma Putri
13424053	Ahmad Zaky Robbani
13524063	Marcel Luther Sitorus
13524081	Alya Nur Rahmah

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132
2026**

DAFTAR ISI

DAFTAR ISI.....	2
TEORI SINGKAT.....	3
1.1. Context-Free Grammar (CFG).....	3
PERANCANGAN DAN IMPLEMENTASI.....	4
2.1. Perancangan.....	4
2.2. Perancangan Aturan Produksi.....	5
2.3. Perancangan Analisis Sintaks (Parser).....	9
2.4. Implementasi.....	11
2.4.1. Alur Program.....	11
2.4.2. Kode Program Utama.....	12
2.4.3. Kode Kelas Parse Tree.....	16
2.4.4. Kode Kelas Parser.....	22
PENGUJIAN.....	33
3.1. Pengujian Pertama.....	33
3.2. Pengujian Kedua.....	34
3.3. Pengujian Ketiga.....	37
3.4. Pengujian Keempat.....	43
3.5. Pengujian Kelima.....	47
3.6. Pengujian Keenam.....	53
3.7. Pengujian Ketujuh.....	61
3.8. Pengujian Kedelapan.....	68
3.9. Pengujian Kesembilan.....	72
3.10. Pengujian Kesepuluh.....	82
3.11. Pengujian Kesebelas.....	99
3.12. Pengujian Keduabelas.....	100
3.13. Pengujian Ketigabelas.....	103
3.14. Pengujian Keempatbelas.....	109
3.15. Pengujian Kelimabelas.....	110
3.16. Pengujian Keenambelas.....	111
3.17. Pengujian Ketujuhbelas.....	113
3.18. Pengujian Kedelapanbelas.....	114
3.19. Pengujian Kesembilanbelas.....	117
3.20. Pengujian Keduapuluh.....	119
3.21. Pengujian Keduapuluhsatu.....	121
3.22. Pengujian Keduapuluhdua.....	123
3.23. Pengujian Keduapuluhtiga.....	127
3.24. Pengujian Keduapuluhempat.....	130
3.25. Pengujian Keduapuluhlima.....	132
3.26. Pengujian Keduapuluhenam.....	133
3.27. Pengujian Keduapuluhtujuh.....	135

3.28. Pengujian Otomatis.....	136
BAB IV.....	137
KESIMPULAN DAN SARAN.....	137
4.1. Kesimpulan.....	137
4.2. Saran.....	137
LAMPIRAN.....	138
DAFTAR PUSTAKA.....	139

BAB I

TEORI SINGKAT

1.1. Context-Free Grammar (CFG)

BAB II

PERANCANGAN DAN IMPLEMENTASI

2.1. Perancangan

Program pada Milestone 2 dirancang sebagai bagian dari pipeline compiler Arion yang menghubungkan tahap lexical analysis dan syntax analysis. Secara umum, program menerima input berupa source code Arion berekstensi .txt, kemudian melakukan tokenisasi menggunakan lexer dari Milestone 1, dan setelah itu menjalankan parser untuk menghasilkan parse tree. Selain itu, pada implementasi terbaru program juga menyediakan mode khusus agar syntax analyzer dapat diuji secara mandiri sesuai kebutuhan penilaian Milestone 2.

Program dimulai dengan membaca argumen command line yang diberikan oleh pengguna. Argumen utama dapat berupa file source code Arion atau file token hasil lexical analysis. Jika pengguna memberikan file source code, program akan menjalankan lexer terlebih dahulu untuk menghasilkan daftar token. Jika pengguna menggunakan mode --from-tokens, program akan langsung membaca daftar token dari file .txt hasil Milestone 1 tanpa melakukan proses lexical analysis ulang. Fitur ini dibuat agar parser dapat diuji sesuai spesifikasi Milestone 2, yaitu menerima masukan berupa list token dan menghasilkan keluaran berupa parse tree.

Setelah argumen diproses, program melakukan validasi terhadap file input. Jika file input tidak ditemukan atau tidak dapat dibuka, program akan menampilkan pesan error dan menghentikan eksekusi. Program juga menentukan lokasi file output untuk token dan parse tree. Jika pengguna tidak memberikan path output secara eksplisit, program akan membuat path output default pada folder test/milestone2/output/. Direktori output akan dibuat secara otomatis apabila belum tersedia.

Apabila input yang diberikan berupa source code, tahap pertama yang dijalankan adalah lexical analysis. Pada tahap ini, program menginisialisasi DFA dari file konfigurasi lexer, kemudian lexer membaca source code karakter demi karakter. Setiap karakter diproses oleh DFA untuk membentuk token sesuai prinsip longest match. Token yang dihasilkan akan disimpan dalam daftar token dan dapat dituliskan ke file output token apabila diperlukan.

Sebelum masuk ke parser, program melakukan penyaringan terhadap token hasil lexer. Token komentar diabaikan karena komentar tidak memengaruhi struktur sintaks program. Jika ditemukan token UNKNOWN, program menganggapnya sebagai lexical error dan proses parsing tidak dilanjutkan. Dengan demikian, parser hanya menerima token yang valid secara leksikal.

Jika pengguna menggunakan mode --from-tokens, program melewati tahap lexical analysis dan langsung membaca token dari file token. Setiap baris pada file token akan dikonversi kembali menjadi objek token yang dapat diproses oleh parser. Mode ini membantu membuktikan bahwa parser dapat menerima output dari Milestone 1 sebagai input Milestone 2.

Tahap berikutnya adalah syntax analysis. Parser menerima daftar token valid dan memprosesnya menggunakan metode recursive descent. Setiap nonterminal pada grammar direpresentasikan oleh fungsi parser tersendiri, misalnya parseProgram, parseProgramHeader, parseDeclarationPart,

parseStatement, parseExpression, parseTerm, dan parseFactor. Parser memulai proses dari simbol awal <program>, kemudian secara rekursif memeriksa apakah urutan token sesuai dengan aturan grammar bahasa Arion.

Selama proses parsing, parser juga membangun parse tree. Node parse tree terdiri atas node nonterminal dan terminal. Node nonterminal merepresentasikan aturan grammar seperti <program>, <compound-statement>, dan <expression>, sedangkan node terminal merepresentasikan token aktual seperti programsy, ident, semicolon, beginsy, dan sebagainya. Jika seluruh token dapat dikonsumsi sesuai grammar, parse tree akan dicetak ke terminal dan/atau disimpan ke file .txt.

Pada implementasi terbaru, program menyediakan mode --parse-only. Mode ini digunakan untuk menjalankan pipeline hanya sampai tahap syntax analysis. Dengan mode ini, program akan berhenti setelah parse tree berhasil dibuat dan tidak melanjutkan proses ke semantic analysis atau intermediate code generation. Fitur ini penting karena Milestone 2 dinilai secara khusus pada kemampuan parser dalam membaca token, memvalidasi grammar, dan menghasilkan parse tree.

Parser juga dilengkapi dengan error handling menggunakan ParseException. Jika parser menemukan token yang tidak sesuai dengan grammar, program akan menampilkan pesan error yang menjelaskan token yang diharapkan dan token yang ditemukan. Misalnya, jika struktur while atau for tidak menggunakan compound-statement setelah keyword do, parser akan menolak input tersebut dan menampilkan syntax error. Hal ini sesuai dengan revisi grammar terbaru yang mewajibkan body while dan for berbentuk begin ... end.

Dengan alur tersebut, program Milestone 2 dapat digunakan dalam dua skenario utama. Pertama, sebagai pipeline dari source code Arion ke parse tree melalui lexer dan parser. Kedua, sebagai parser mandiri yang menerima file token hasil Milestone 1 melalui mode --from-tokens. Kedua skenario ini memastikan bahwa syntax analyzer memenuhi kebutuhan spesifikasi, yaitu menggunakan recursive descent, membaca token hasil lexical analysis, memvalidasi struktur program berdasarkan grammar Arion, serta menghasilkan parse tree yang terformat dengan jelas.

2.2. Perancangan Aturan Produksi

Perancangan aturan produksi ini dibuat untuk merepresentasikan alur yang diimplementasikan pada file parser.cpp. Terdapat beberapa transformasi yang diterapkan pada aturan produksi, terutama aturan produksi yang mengandung variabel sisi kiri pada sisi kanannya untuk menghindari *infinite loop*. Berikut adalah perancangan aturan produksi final yang diterapkan pada parser.cpp.

```
Program
-> ProgramHeader Block PERIOD

ProgramHeader
-> PROGRAMSY IDENT SEMICOLON
| PROGRAMSY IDENT LPARENT IdentifierList RPARENT SEMICOLON
```

```

IdentifierList
    -> IDENT
    | IDENT COMMA IdentifierList

Block
    -> DeclarationPart CompoundStatement

DeclarationPart
    -> { ConstDeclaration | TypeDeclaration | VarDeclaration |
SubprogramDeclarationPart }

ConstDeclaration
    -> CONSTSY ConstDefinition SEMICOLON { ConstDefinition SEMICOLON }

ConstDefinition
    -> IDENT BECOMES Constant

TypeDeclaration
    -> TYPESY TypeDefinition SEMICOLON { TypeDefinition SEMICOLON }

TypeDefinition
    -> IDENT EQL Type

VarDeclaration
    -> VARSY VariableDeclaration SEMICOLON { VariableDeclaration SEMICOLON }

VariableDeclaration
    -> IdentifierList COLON Type

Type
    -> IDENT
    | ArrayType
    | RecordType
    | Enumerated

ArrayType
    -> ARRAYSY LBRACK Range { COMMA Range } RBRACK OFSY Type

Range
    -> Constant PERIOD PERIOD Constant

Enumerated
    -> LPARENT IdentifierList RPARENT

RecordType
    -> RECORDSY FieldList ENDSY

FieldList
    -> VariableDeclaration { SEMICOLON VariableDeclaration } [ SEMICOLON ]

SubprogramDeclarationPart
    -> SubprogramDeclaration SEMICOLON { SubprogramDeclaration SEMICOLON }

SubprogramDeclaration
    -> ProcedureDeclaration

```

```

    | FunctionDeclaration

ProcedureDeclaration
    -> ProcedureHeading SEMICOLON Block

FunctionDeclaration
    -> FunctionHeading SEMICOLON Block

ProcedureHeading
    -> PROCEDURESY IDENT
    | PROCEDURESY IDENT FormalParameterList

FunctionHeading
    -> FUNCTIONSY IDENT COLON IDENT
    | FUNCTIONSY IDENT FormalParameterList COLON IDENT

FormalParameterList
    -> LPARENT ParameterGroup { SEMICOLON ParameterGroup } RPARENT

ParameterGroup
    -> IdentifierList COLON IDENT

CompoundStatement
    -> BEGINSY StatementList ENDSY

StatementList
    -> Statement { SEMICOLON Statement }

Statement
    -> AssignmentStatement
    | ProcedureFunctionCall
    | CompoundStatement
    | IfStatement
    | CaseStatement
    | RepeatStatement
    | WhileStatement
    | ForStatement
    | Empty

Empty
    -> ε

AssignmentStatement
    -> Variable BECOMES Expression

ProcedureFunctionCall
    -> IDENT
    | IDENT LPARENT [ ParameterList ] RPARENT
    | Variable

IfStatement
    -> IFSY Expression THENSY Statement
    | IFSY Expression THENSY Statement ELSESY Statement

CaseStatement
    -> CASESY Expression OFSY CaseBlock { SEMICOLON CaseBlock } [ SEMICOLON ]

```


ENDSY

CaseBlock

-> Constant COLON Statement

RepeatStatement

-> REPEATSY StatementList UNTILSY Expression

WhileStatement

-> WHILESY Expression DOSY CompoundStatement SEMICOLON

ForStatement

-> FORSY IDENT BECOMES Expression TOSY Expression DOSY CompoundStatement

SEMICOLON

| FORSY IDENT BECOMES Expression DOWNTOSY Expression DOSY CompoundStatement

SEMICOLON

ParameterList

-> Expression { COMMA Expression }

Expression

-> SimpleExpression

| SimpleExpression RelationalOperator SimpleExpression

RelationalOperator

-> EQL

| NEQ

| LSS

| LEQ

| GTR

| GEQ

SimpleExpression

-> Term { AdditiveOperator Term }

AdditiveOperator

-> PLUS

| MINUS

| ORSY

Term

-> Factor { MultiplicativeOperator Factor }

MultiplicativeOperator

-> TIMES

| IDIV

| RDIV

| IMOD

| ANDSY

Factor

-> LPARENT Expression RPARENT

| NOTSY Factor

| Variable

| ProcedureFunctionCall

| Constant

```

Variable
  -> IDENT { Selector }

Selector
  -> LBRACK IndexList RBRACK
  | PERIOD IDENT

IndexList
  -> Expression { COMMA Expression }

Constant
  -> [ PLUS | MINUS ] ConstantValue

ConstantValue
  -> IDENT
  | INTCON
  | REALCON
  | CHARCON
  | STRING

```

Keterangan:

1. { X } artinya X dapat berulang sebanyak nol sampai beberapa kali.
2. [X] artinya X adalah opsional.
3. ϵ artinya aturan produksi untuk kasus token kosong.
4. *Starting Variable* dari aturan produksi tersebut adalah variabel Program

2.3. Perancangan Analisis Sintaks (Parser)

Analisis sintaks pada Milestone 2 dirancang untuk memeriksa apakah urutan token hasil lexical analysis sesuai dengan aturan grammar bahasa Arion. Parser yang digunakan adalah parser berbasis recursive descent, yaitu metode parsing top-down yang merepresentasikan setiap nonterminal pada grammar sebagai fungsi parsing tersendiri.

Masukan utama parser adalah daftar token hasil lexical analysis. Token tersebut dapat diperoleh melalui dua cara. Pertama, program membaca source code Arion, menjalankan lexer, kemudian langsung meneruskan daftar token valid ke parser. Kedua, parser dapat membaca langsung file token hasil Milestone 1 melalui mode --from-tokens. Mode ini membuat syntax analyzer dapat diuji secara mandiri menggunakan file token .txt.

Sebelum proses parsing dilakukan, token komentar (COMMENT) diabaikan karena komentar tidak memengaruhi struktur sintaks program. Jika ditemukan token UNKNOWN, program menganggapnya sebagai lexical error dan proses parsing tidak dilanjutkan. Dengan demikian, parser hanya menerima token yang valid secara leksikal.

Parser memulai analisis dari simbol awal <program>. Fungsi parseProgram() akan memanggil fungsi parsing lain sesuai struktur grammar, seperti parseProgramHeader(), parseDeclarationPart(), dan parseCompoundStatement(). Selanjutnya, setiap fungsi parsing akan memeriksa token yang sedang dibaca menggunakan fungsi bantu seperti current(), lookAhead(), check(), consume(), dan expect().

Setiap nonterminal pada grammar diimplementasikan sebagai fungsi tersendiri. Misalnya, aturan ekspresi diproses oleh `parseExpression()`, `parseSimpleExpression()`, `parseTerm()`, dan `parseFactor()`. Struktur ini menjaga prioritas operator, sehingga operator perkalian dan pembagian diproses lebih dahulu daripada operator penjumlahan dan pengurangan. Statement diproses melalui `parseStatement()`, yang kemudian melakukan dispatch ke bentuk statement yang sesuai, seperti assignment, procedure/function call, if statement, case statement, repeat statement, while statement, for statement, atau compound statement.

Selama proses parsing, parser membangun parse tree. Setiap node pada parse tree dapat berupa node terminal atau nonterminal. Node nonterminal merepresentasikan aturan grammar seperti `<program>`, `<statement>`, `<expression>`, dan `<factor>`, sedangkan node terminal merepresentasikan token aktual seperti `programs`, `ident`, `semicolon`, dan `beginsy`.

Jika token yang ditemukan tidak sesuai dengan grammar, parser akan melempar `ParseException`. Pesan error tersebut memuat token yang diharapkan dan token yang ditemukan. Error handling ini membuat program dapat menolak input yang tidak valid tanpa menyebabkan crash.

Pada implementasi terbaru, parser juga memperketat aturan `while` dan `for`. Body setelah token `dosy` wajib berupa `compound-statement`, yaitu blok `begin ... end`. Dengan demikian, bentuk seperti `while x < 5 do x := x + 1;` akan ditolak, sedangkan bentuk yang valid adalah `while x < 5 do begin x := x + 1; end;`.

Dengan rancangan ini, parser memenuhi kebutuhan Milestone 2, yaitu menggunakan recursive descent, membaca token hasil lexer, memvalidasi struktur program berdasarkan grammar Arion, menghasilkan parse tree, dan memberikan pesan error sintaks yang informatif.

2.4. Implementasi

2.4.1. Alur Program

Program pada Milestone 2 berfungsi sebagai syntax analyzer untuk bahasa Arion. Program ini berada setelah tahap lexical analysis dan bertugas memeriksa apakah urutan token yang dihasilkan lexer sudah sesuai dengan grammar bahasa Arion. Hasil utama dari tahap ini adalah parse tree yang merepresentasikan struktur sintaks program secara hierarkis.

Secara umum, alur program dimulai dari pembacaan argumen command line. Pengguna dapat memberikan input berupa source code Arion atau file token hasil Milestone 1. Jika input berupa source code, program akan menjalankan lexer terlebih dahulu untuk menghasilkan daftar token. Jika pengguna menggunakan mode `--from-tokens`, program akan langsung membaca daftar token dari file `.txt` tanpa menjalankan proses lexical analysis ulang. Mode ini disediakan agar parser dapat diuji secara mandiri menggunakan output dari Milestone 1.

Setelah argumen diproses, program melakukan validasi terhadap file input. Jika file input tidak dapat ditemukan atau tidak dapat dibuka, program akan menampilkan pesan error melalui `std::cerr` dan menghentikan eksekusi dengan exit code non-zero. Program juga menentukan lokasi file output untuk token dan parse tree. Jika pengguna tidak memberikan path output secara eksplisit, program akan menggunakan path default pada folder pengujian Milestone 2.

Apabila input yang diberikan berupa source code, tahap pertama yang dijalankan adalah lexical analysis. Program menginisialisasi DFA dari file konfigurasi `config/config_lexer.txt`, kemudian membuat objek Lexer untuk membaca source code karakter demi karakter. Lexer akan menghasilkan token berdasarkan transisi DFA dan menuliskan token tersebut ke file output token. Token yang memiliki nilai lexeme, seperti `ident`, `intcon`, `realcon`, `charcon`, dan `string`, akan ditampilkan bersama nilainya.

Sebelum token dikirim ke parser, program melakukan penyaringan terhadap token hasil lexer. Token komentar (`COMMENT`) diabaikan karena komentar tidak memengaruhi struktur sintaks program. Jika ditemukan token `UNKNOWN`, program menganggapnya sebagai lexical error sehingga proses parsing tidak dilanjutkan. Dengan demikian, parser hanya menerima token yang sudah valid secara leksikal.

Jika mode `--from-tokens` digunakan, program melewati tahap lexical analysis. File token yang diberikan akan dibaca dan dikonversi kembali menjadi daftar objek Token. Daftar token tersebut kemudian langsung digunakan sebagai input parser. Fitur ini menunjukkan bahwa parser tidak bergantung secara langsung pada source code, melainkan dapat menerima daftar token hasil lexical analyzer sebagaimana diminta pada Milestone 2.

Tahap berikutnya adalah syntax analysis. Parser menerima daftar token valid dan memprosesnya menggunakan metode recursive descent. Setiap nonterminal pada grammar direpresentasikan oleh fungsi parsing tersendiri, seperti `parseProgram`, `parseProgramHeader`, `parseDeclarationPart`, `parseCompoundStatement`, `parseStatement`, `parseExpression`, `parseTerm`, dan `parseFactor`. Proses

parsing dimulai dari simbol awal <program>, kemudian dilanjutkan secara rekursif sesuai aturan produksi grammar Arion.

Selama proses parsing berlangsung, parser membangun parse tree. Parse tree terdiri atas node nonterminal dan terminal. Node nonterminal merepresentasikan struktur grammar seperti <program>, <declaration-part>, <compound-statement>, <statement>, dan <expression>. Sementara itu, node terminal merepresentasikan token aktual seperti programsy, ident, semicolon, beginsy, endsy, dan sebagainya.

Apabila urutan token sesuai dengan grammar, parser akan menghasilkan parse tree dan menyimpannya ke file output. Parse tree juga dapat ditampilkan ke terminal agar proses parsing dapat diperiksa secara langsung. Pada implementasi terbaru, program menyediakan mode --parse-only untuk menjalankan pipeline hanya sampai tahap syntax analysis. Dengan mode ini, program berhenti setelah parse tree berhasil dibentuk dan tidak melanjutkan proses ke semantic analysis atau tahap berikutnya.

Parser juga dilengkapi dengan error handling menggunakan ParseException. Jika token yang ditemukan tidak sesuai dengan token yang diharapkan oleh grammar, parser akan menghentikan proses parsing dan menampilkan pesan error yang menjelaskan token yang diharapkan serta token yang ditemukan. Error ini juga menghasilkan exit code non-zero sehingga dapat digunakan dalam pengujian otomatis.

Pada implementasi terbaru, grammar untuk while dan for telah disesuaikan dengan revisi spesifikasi. Body setelah keyword do wajib berupa compound-statement, yaitu blok begin ... end. Oleh karena itu, bentuk seperti while x < 5 do x := x + 1; akan ditolak oleh parser, sedangkan bentuk while x < 5 do begin x := x + 1; end; akan diterima.

Dengan alur tersebut, program Milestone 2 dapat digunakan dalam dua skenario utama. Pertama, sebagai pipeline dari source code Arion menuju token dan parse tree. Kedua, sebagai parser mandiri yang menerima file token hasil Milestone 1 melalui mode --from-tokens. Implementasi ini memastikan bahwa syntax analyzer memenuhi kebutuhan utama Milestone 2, yaitu membaca token hasil lexer, memvalidasi struktur program berdasarkan grammar, menggunakan recursive descent, menghasilkan parse tree, serta menangani syntax error secara informatif.

2.4.2. Kode Program Utama

Implementasi
<pre>#include <filesystem> #include <fstream> #include <iostream> #include <memory> #include <string> #include <vector> #include "lexer/dfa.hpp"</pre>

```

#include "lexer/lexer.hpp"
#include "lexer/lexer_exception.hpp"
#include "lexer/token.hpp"
#include "parser/parser.hpp"
#include "parser/parse_tree.hpp"

namespace fs = std::filesystem;

struct Options {
    std::string inputFile;
    std::string tokenOutputFile;
    std::string parseTreeOutputFile;
    bool parseOnly = false;
    bool fromTokens = false;
};

void ensureParentDirectoryExists(const std::string& filename) {
    fs::path path(filename);

    if (path.has_parent_path()) {
        fs::create_directories(path.parent_path());
    }
}

bool isIgnoredByParser(const Token& token) {
    return token.get_type_name() == "COMMENT";
}

bool isLexicalErrorToken(const Token& token) {
    return token.get_type_name() == "UNKNOWN";
}

std::vector<Token> runLexer(const std::string& inputFile,
                           const std::string& tokenOutputFile) {
    std::ifstream source(inputFile);

    if (!source.is_open()) {
        throw std::runtime_error("Gagal membuka file input: " + inputFile);
    }

    ensureParentDirectoryExists(tokenOutputFile);
    std::ofstream tokenOutput(tokenOutputFile);

    if (!tokenOutput.is_open()) {
        throw std::runtime_error("Gagal membuka file output token: " +
tokenOutputFile);
    }

    auto dfa = std::make_shared<DFA>();
    dfa->loadConfig("config/config_lexer.txt");

    Lexer lexer(source, dfa, &tokenOutput);

    while (!lexer.eof()) {
        lexer.process_next_token();
    }

    std::vector<Token> parserTokens;

    for (const Token& token : lexer.getResult()) {
        if (isIgnoredByParser(token)) {
            continue;
        }
    }
}

```

```

    }

    if (isLexicalErrorToken(token)) {
        throw std::runtime_error("Lexical error: token UNKNOWN ditemukan.");
    }

    parserTokens.push_back(token);
}

return parserTokens;
}

void saveParseTree(const parse_tree::NodePtr& root,
                  const std::string& outputFile) {
    ensureParentDirectoryExists(outputFile);
    std::ofstream out(outputFile);

    if (!out.is_open()) {
        throw std::runtime_error("Gagal membuka file output parse tree: " +
outputFile);
    }

    parse_tree::printTree(root.get(), std::cout);
    parse_tree::printTree(root.get(), out);
}

int main(int argc, char* argv[]) {
    try {
        Options options;

        if (argc < 2) {
            throw std::runtime_error("Input file belum diberikan.");
        }

        options.inputFile = argv[1];

        for (int i = 2; i < argc; ++i) {
            std::string arg = argv[i];

            if (arg == "--save-tokens" || arg == "-o") {
                options.tokenOutputFile = argv[++i];
            } else if (arg == "--save-parse-tree" || arg == "-p") {
                options.parseTreeOutputFile = argv[++i];
            } else if (arg == "--parse-only") {
                options.parseOnly = true;
            } else if (arg == "--from-tokens") {
                options.fromTokens = true;
                options.inputFile = argv[++i];
            } else {
                throw std::runtime_error("Argumen tidak dikenal: " + arg);
            }
        }

        std::vector<Token> tokens;

        if (options.fromTokens) {
            tokens = readTokensFromFile(options.inputFile);
        } else {
            tokens = runLexer(options.inputFile, options.tokenOutputFile);
        }

        Parser parser(tokens);

```

```

        parse_tree::NodePtr root = parser.parse();

        saveParseTree(root, options.parseTreeOutputFile);

        std::cout << "Parsing selesai.\n";

        if (options.parseOnly) {
            return 0;
        }

        return 0;
    } catch (const LexerException& e) {
        std::cerr << e.full_message() << "\n";
        return 1;
    } catch (const ParseException& e) {
        std::cerr << e.full_message() << "\n";
        return 1;
    } catch (const std::exception& e) {
        std::cerr << "ERROR: " << e.what() << "\n";
        return 1;
    }
}

```

Penjelasan

Program utama berfungsi sebagai driver yang menghubungkan komponen lexical analyzer dan syntax analyzer dalam pipeline compiler Arion. Program menerima input melalui argumen command line, baik berupa source code Arion maupun file token hasil lexical analysis. Selain itu, program menyediakan beberapa opsi tambahan, seperti penentuan file output token, file output parse tree, mode `--lex-only`, mode `--parse-only`, dan mode `--from-tokens`.

Pada tahap awal, program melakukan validasi argumen untuk memastikan bahwa input yang diberikan pengguna sesuai dengan format yang didukung. Jika argumen tidak valid atau file input tidak ditemukan, program akan menampilkan pesan error dan menghentikan eksekusi. Setelah itu, program menentukan lokasi file output. Jika pengguna tidak menentukan path output secara eksplisit, program akan membuat path output default berdasarkan nama file input. Direktori output juga dibuat secara otomatis menggunakan `std::filesystem` apabila belum tersedia.

Jika input yang diberikan berupa source code Arion, program akan menjalankan lexical analysis terlebih dahulu. Pada tahap ini, program membuka file input, menginisialisasi objek DFA, lalu memuat konfigurasi state dan transisi dari file eksternal `config/config_lexer.txt`. DFA tersebut digunakan oleh objek Lexer untuk membaca source code karakter demi karakter. Proses tokenisasi dilakukan secara iteratif menggunakan fungsi `process_next_token()` sampai seluruh isi file selesai diproses. Token yang dihasilkan kemudian disimpan dalam daftar token dan dapat dituliskan ke file output token.

Setelah proses lexical analysis selesai, seluruh token hasil lexer diperiksa kembali sebelum masuk ke parser. Token komentar (COMMENT) diabaikan karena tidak memengaruhi struktur sintaks program. Sebaliknya, token UNKNOWN dianggap sebagai lexical error, sehingga program langsung menghentikan proses parsing. Dengan cara ini, parser hanya menerima token yang sudah valid secara leksikal.

Selain menerima source code, program juga mendukung mode `--from-tokens`. Mode ini memungkinkan parser membaca langsung file token hasil Milestone 1 tanpa menjalankan lexer ulang. Fitur ini penting

karena pada spesifikasi Milestone 2, masukan utama syntax analyzer adalah list token hasil lexical analysis. Dengan demikian, parser dapat diuji secara mandiri menggunakan file token sebagai input.

Tahap berikutnya adalah syntax analysis. Parser menerima vector<Token> yang berisi token valid, kemudian memprosesnya menggunakan metode recursive descent. Setiap nonterminal pada grammar direpresentasikan oleh fungsi parser tersendiri, seperti parseProgram, parseDeclarationPart, parseStatement, parseExpression, parseTerm, dan parseFactor. Parser memulai proses dari simbol awal <program>, lalu memeriksa urutan token secara rekursif berdasarkan aturan produksi grammar Arion.

Jika proses parsing berhasil, parser menghasilkan parse tree yang merepresentasikan struktur sintaks program. Parse tree tersebut dapat dicetak ke terminal dan disimpan ke file output parse tree. Program juga menyediakan mode --parse-only, yaitu mode khusus untuk menghentikan pipeline setelah tahap syntax analysis. Dengan mode ini, program hanya menjalankan lexer dan parser, lalu menghasilkan parse tree tanpa melanjutkan ke semantic analysis. Mode ini digunakan agar Milestone 2 dapat diuji secara terpisah sesuai fokus penilaian parser.

Program juga dilengkapi dengan exception handling menggunakan blok try-catch. Error yang mungkin terjadi, seperti LexerException, ParseException, dan exception umum lainnya, akan ditangani dengan menampilkan pesan error yang informatif. Pada tahap parsing, jika token yang ditemukan tidak sesuai dengan grammar, program akan menampilkan token yang diharapkan dan token yang ditemukan. Hal ini membantu proses debugging serta menunjukkan bahwa parser mampu menangani kesalahan sintaks tanpa menyebabkan program crash.

Dengan alur tersebut, program Milestone 2 tidak hanya dapat dijalankan sebagai bagian dari pipeline source code menuju parse tree, tetapi juga dapat diuji sebagai parser mandiri melalui mode --from-tokens dan --parse-only. Implementasi ini memperkuat kesesuaian program terhadap spesifikasi Milestone 2, terutama pada aspek penerimaan input token, penggunaan recursive descent, penghasilan parse tree, dan error handling.

2.4.3. Kode Kelas Parse Tree

Header

```
// Beberapa kode tidak dituliskan semua untuk mempersingkat
// Untuk lengkapnya bisa dilihat di repo github
#ifndef PARSE_TREE_HPP
#define PARSE_TREE_HPP

#include "../lexer/token.hpp"

#include <iosfwd>
#include <memory>
#include <string>
#include <vector>

namespace parse_tree {

enum class NodeKind {
    Terminal,
    NonTerminal,
```

```

    Error
};

enum class NonTerminal {
    // Berisi jenis node nonterminal
};

class Node {
public:
    Node(NodeKind kind, std::string label, std::string value = "", int line = -1,
        int column = -1);

    // Takes ownership of raw child pointers produced by
    createTerminal/createNonTerminal.
    Node* addChild(Node* child);
    Node* addChild(std::unique_ptr<Node> child);

    NodeKind getKind() const;
    bool isTerminal() const;
    bool isNonTerminal() const;
    bool isError() const;
    const std::string& getLabel() const;
    const std::string& getValue() const;
    int getLine() const;
    int getColumn() const;
    const std::vector<std::unique_ptr<Node>>& getChildren() const;

    std::string toString() const;
    void print(std::ostream& out, int depth = 0) const;

private:
    NodeKind kind;
    std::string label;
    std::string value;
    int line;
    int column;
    std::vector<std::unique_ptr<Node>> children;
};

using NodePtr = std::unique_ptr<Node>;

const char* toString(NonTerminal symbol);

NodePtr makeNonTerminal(NonTerminal symbol);
NodePtr makeNonTerminal(const std::string& label);
NodePtr makeTerminal(const std::string& tokenType, const std::string& lexeme = "",
    int line = -1, int column = -1);
NodePtr makeTerminal(const Token& token);
NodePtr makeError(const std::string& message, int line = -1, int column = -1);

Node* createNonTerminal(NonTerminal symbol);
Node* createNonTerminal(const std::string& label);
Node* createTerminal(const std::string& tokenType, const std::string& lexeme = "",
    int line = -1, int column = -1);
Node* createTerminal(const Token& token);
Node* createError(const std::string& message, int line = -1, int column = -1);

void printTree(const Node* root, std::ostream& out);

} // namespace parse_tree

#endif

```

Implementasi

```
// Beberapa kode tidak dituliskan semua untuk mempersingkat
// Untuk lengkapnya bisa dilihat di repo github

using NodePtr = std::unique_ptr<Node>;

Node::Node(NodeKind kind, std::string label, std::string value, int line, int
column) : kind(kind), label(std::move(label)), value(std::move(value)),
line(line), column(column) {}
// Constructor parse tree node

Node* Node::addChild(Node* child) {
    // Menambahkan child node
    if (child == nullptr) {
        return nullptr;
    }

    children.emplace_back(child);
    return children.back().get();
}

std::string Node::toString() const {
    // Format non-terminal
    if (isNonTerminal()) {
        return "<" + label + ">";
    }

    // Format error node
    if (isError()) {
        return "<error: " + value + ">";
    }

    // Format terminal dengan lexeme
    if (!value.empty()) {
        return label + " (" + value + ")";
    }

    return label;
}

void Node::print(std::ostream& out,
                int depth) const {

    // Memberikan indentasi sesuai depth
    for (int i = 0; i < depth; ++i) {
        out << "  ";
    }

    out << toString();

    // Menampilkan posisi token
    if (line >= 0 && column >= 0) {
        out << " [" << line
```

```

        << ":" << column << "];";
    }

    out << '\n';

    // Print child secara rekursif
    for (const auto& child : children) {
        child->print(out, depth + 1);
    }
}

const char* toString(NonTerminal symbol) {
    // Mengubah enum menjadi string grammar
    switch (symbol) {

        case NonTerminal::Program:
            return "program";

        case NonTerminal::Statement:
            return "statement";

        case NonTerminal::Expression:
            return "expression";

        // ... non-terminal lainnya ...

        default:
            throw std::logic_error(
                "Unhandled non-terminal symbol"
            );
    }
}

NodePtr makeNonTerminal(
    NonTerminal symbol) {

    // Membuat node non-terminal
    return std::make_unique<Node>(
        NodeKind::NonTerminal,
        toString(symbol)
    );
}

NodePtr makeTerminal(
    const Token& token) {

    // Membuat node terminal dari token lexer
    std::string label =
        token.get_type_name();

    std::transform(
        label.begin(),
        label.end(),
        label.begin(),
        [](unsigned char c) {
            return std::tolower(c);
        }
    );
}

```

```

        }
    };

    return makeTerminal(
        label,
        token.get_value()
    );
}

NodePtr makeError(
    const std::string& message,
    int line,
    int column) {

    // Membuat node error parser
    return std::make_unique<Node>(
        NodeKind::Error,
        "error",
        message,
        line,
        column
    );
}

Node* createNonTerminal(
    NonTerminal symbol) {

    // Membuat raw pointer non-terminal
    return makeNonTerminal(
        symbol
    ).release();
}

Node* createTerminal(
    const Token& token) {

    // Membuat raw pointer terminal
    return makeTerminal(
        token
    ).release();
}

Node* createError(
    const std::string& message,
    int line,
    int column) {

    // Membuat raw pointer error node
    return makeError(
        message,
        line,
        column
    ).release();
}

void printTree(const Node* root,

```

```

        std::ostream& out) {

    // Print seluruh parse tree
    if (root == nullptr) {
        out << "<empty tree>\n";
        return;
    }

    root->print(out);
}

/*
Fungsi lain yang digunakan:

- bool isTerminal()
    Mengecek apakah node terminal

- bool isNonTerminal()
    Mengecek apakah node non-terminal

- bool isError()
    Mengecek apakah node error

- getLabel(), getValue()
    Mengambil informasi node

- getLine(), getColumn()
    Mengambil posisi token

- getChildren()
    Mengambil child node

- makeNonTerminal(const std::string&)
    Membuat non-terminal dari string

- makeTerminal(...)
    Membuat terminal manual
*/

```

Penjelasan

Kelas Parse Tree digunakan untuk merepresentasikan struktur sintaks program dalam bentuk pohon hasil parsing. Setiap node dibedakan menjadi Terminal, NonTerminal, dan Error menggunakan enum NodeKind. Kelas Node menjadi struktur utama parse tree yang menyimpan label node, nilai token, dan daftar child node. Penyimpanan child menggunakan std::unique_ptr agar pengelolaan memori dilakukan otomatis dan lebih aman. Fungsi addChild() digunakan untuk menambahkan node anak ke dalam parse tree. Fungsi toString() mengubah node menjadi format string yang mudah dibaca, sedangkan fungsi print() digunakan untuk mencetak parse tree secara rekursif dengan indentasi sesuai kedalaman node.

Selain itu, helper function seperti makeNonTerminal() dan makeTerminal() digunakan untuk mempermudah pembuatan node selama proses recursive descent parsing berlangsung. Setelah parsing selesai, seluruh struktur parse tree dapat ditampilkan menggunakan fungsi printTree().

2.4.4. Kode Kelas Parser

Header

```
// Beberapa kode tidak dituliskan semua untuk mempersingkat
// Untuk lengkapnya bisa dilihat di repo github

#ifndef PARSER_HPP
#define PARSER_HPP

#include "../lexer/token.hpp"
#include "parse_tree.hpp"

#include <stdexcept>
#include <string>
#include <vector>

class ParseException : public std::runtime_error {
public:
    int line;
    int column;

    ParseException(const std::string& message, int line = -1, int column = -1)
        : std::runtime_error(message), line(line), column(column) {}

    std::string full_message() const {
        if (line >= 0 && column >= 0) {
            return "[ParseError " + std::to_string(line) + ":" +
                std::to_string(column) + "] " + what();
        }
        return std::string("[ParseError] ") + what();
    }
};

class Parser {
public:
    explicit Parser(const std::vector<Token>& tokens);

    parse_tree::NodePtr parse();

private:
    const std::vector<Token>& tokens;
    size_t pos;
    bool atEnd() const;
    const Token& current() const;
    const Token& lookAhead(size_t offset = 1) const;
    bool check(const std::string& typeName) const;
    bool checkAny(std::initializer_list<std::string> types) const;

    Token consume();
    Token expect(const std::string& typeName);
    parse_tree::NodePtr termNode();
    parse_tree::NodePtr expectNode(const std::string& typeName);

    bool isConstantStart() const;
    bool isStatementStart() const;
    bool isTypeStart() const;
    bool isDeclarationStart() const;
    bool isRelationalOp() const;
    bool isAdditiveOp() const;
    bool isMultiplicativeOp() const;
```

```

    parse_tree::NodePtr parseProgram();
    parse_tree::NodePtr parseProgramHeader();
    parse_tree::NodePtr parseIdentifierList();

    parse_tree::NodePtr parseBlock();
    parse_tree::NodePtr parseDeclarationPart();

    parse_tree::NodePtr parseConstDeclaration();
    parse_tree::NodePtr parseConstDefinition();

    parse_tree::NodePtr parseTypeDeclaration();
    parse_tree::NodePtr parseTypeDefinition();

    parse_tree::NodePtr parseVarDeclaration();
    parse_tree::NodePtr parseFieldPart();

    parse_tree::NodePtr parseType();
    parse_tree::NodePtr parseArrayType();
    parse_tree::NodePtr parseRange();
    parse_tree::NodePtr parseEnumerated();
    parse_tree::NodePtr parseRecordType();
    parse_tree::NodePtr parseFieldList();

    parse_tree::NodePtr parseSubprogramDeclarationPart();
    parse_tree::NodePtr parseSubprogramDeclaration();
    parse_tree::NodePtr parseProcedureDeclaration();
    parse_tree::NodePtr parseFunctionDeclaration();
    parse_tree::NodePtr parseProcedureHeading();
    parse_tree::NodePtr parseFunctionHeading();
    parse_tree::NodePtr parseFormalParameterList();
    parse_tree::NodePtr parseParameterGroup();

    parse_tree::NodePtr parseCompoundStatement();
    parse_tree::NodePtr parseStatementList();
    parse_tree::NodePtr parseStatement();
    parse_tree::NodePtr parseAssignmentStatement(parse_tree::NodePtr varNode);
    parse_tree::NodePtr parseProcedureFunctionCall();
    parse_tree::NodePtr parseIfStatement();
    parse_tree::NodePtr parseCaseStatement();
    parse_tree::NodePtr parseCaseBlock();
    parse_tree::NodePtr parseRepeatStatement();
    parse_tree::NodePtr parseWhileStatement();
    parse_tree::NodePtr parseForStatement();
    parse_tree::NodePtr parseParameterList();

    parse_tree::NodePtr parseExpression();
    parse_tree::NodePtr parseRelationalOperator();
    parse_tree::NodePtr parseSimpleExpression();
    parse_tree::NodePtr parseAdditiveOperator();
    parse_tree::NodePtr parseTerm();
    parse_tree::NodePtr parseMultiplicativeOperator();
    parse_tree::NodePtr parseFactor();

    parse_tree::NodePtr parseVariable();
    parse_tree::NodePtr parseSelector();
    parse_tree::NodePtr parseIndexList();
    parse_tree::NodePtr parseConstant();
};

#endif

```


Implementasi

```
// Beberapa kode tidak dituliskan semua untuk mempersingkat
// Untuk lengkapnya bisa dilihat di repo github

class ParseException : public std::runtime_error {
public:
    int line;
    int column;

    // Constructor error parser
    ParseException(const std::string& message,
                  int line = -1,
                  int column = -1);

    // Format pesan error lengkap
    std::string full_message() const;
};

class Parser {
public:
    // Constructor parser
    explicit Parser(const std::vector<Token>& tokens);

    // Memulai proses parsing
    parse_tree::NodePtr parse();

private:
    const std::vector<Token>& tokens;
    size_t pos;

    // Mengecek akhir token
    bool atEnd() const;

    // Mengambil token saat ini
    const Token& current() const;

    // Melihat token berikutnya
    const Token& lookAhead(size_t offset = 1) const;

    // Mengecek tipe token
    bool check(const std::string& typeName) const;

    // Mengonsumsi token
    Token consume();

    // Memastikan token sesuai grammar
    Token expect(const std::string& typeName);

    // Membuat node terminal
    parse_tree::NodePtr termNode();

    // Parsing program utama
    parse_tree::NodePtr parseProgram();

    // Parsing expression
```

```

    parse_tree::NodePtr parseExpression();

    // Parsing statement
    parse_tree::NodePtr parseStatement();

    // Parsing variable
    parse_tree::NodePtr parseVariable();
};

Parser::Parser(const std::vector<Token>& tokens)
    : tokens(tokens), pos(0) {}
// Constructor parser

bool Parser::atEnd() const {
    // Mengecek apakah token habis
    return pos >= tokens.size();
}

const Token& Parser::current() const {
    // Mengambil token saat ini
    if (atEnd()) {
        static Token eof_token(
            TokenType("EOF"), ""
        );
        return eof_token;
    }

    return tokens[pos];
}

const Token& Parser::lookAhead(
    size_t offset) const {

    // Melihat token selanjutnya
    size_t idx = pos + offset;

    if (idx >= tokens.size()) {
        static Token eof_token(
            TokenType("EOF"), ""
        );
        return eof_token;
    }

    return tokens[idx];
}

bool Parser::check(
    const std::string& typeName) const {

    // Mengecek tipe token
    if (atEnd()) {
        return typeName == "EOF";
    }

    return current().get_type_name()
        == typeName;
}

```

```

}

Token Parser::consume() {
    // Mengambil token dan maju
    if (atEnd()) {
        throw ParseException(
            "Sudah mencapai akhir input"
        );
    }

    return tokens[pos++];
}

Token Parser::expect(
    const std::string& typeName) {

    // Validasi token grammar
    if (!check(typeName)) {

        std::string got =
            atEnd() ? "EOF"
                : current().get_type_name();

        throw ParseException(
            "Diharapkan '" +
            typeName +
            "'", mendapat "'" +
            got + "'"
        );
    }

    return consume();
}

NodePtr Parser::termNode() {
    // Membuat node terminal
    Token tok = consume();
    return makeTerminal(tok);
}

NodePtr Parser::parse() {
    // Entry point parser
    return parseProgram();
}

NodePtr Parser::parseProgram() {
    NodePtr node =
        makeNonTerminal(
            NonTerminal::Program
        );

    // Program -> header declaration statement
    node->addChild(
        parseProgramHeader()
    );
}

```

```

        node->addChild(
            parseDeclarationPart()
        );

        node->addChild(
            parseCompoundStatement()
        );

        node->addChild(
            expectNode("PERIOD")
        );

        return node;
    }

NodePtr Parser::parseProgramHeader() {
    NodePtr node =
        makeNonTerminal(
            NonTerminal::ProgramHeader
        );

    // Parsing:
    // program nama_program;
    node->addChild(
        expectNode("PROGRAMSY")
    );

    node->addChild(
        expectNode("IDENT")
    );

    node->addChild(
        expectNode("SEMICOLON")
    );

    return node;
}

NodePtr Parser::parseStatement() {
    NodePtr node =
        makeNonTerminal(
            NonTerminal::Statement
        );

    // Parsing statement program
    if (check("IDENT")) {

        const std::string& next =
            lookAhead(1).get_type_name();

        // Assignment statement
        if (next == "BECOMES") {

            NodePtr varNode =
                parseVariable();

```

```

        node->addChild(
            parseAssignmentStatement(
                std::move(varNode)
            )
        );

    } else {
        // Procedure/function call
        node->addChild(
            parseProcedureFunctionCall()
        );
    }

} else if (check("IFSY")) {

    node->addChild(
        parseIfStatement()
    );

} else if (check("WHILESY")) {

    node->addChild(
        parseWhileStatement()
    );

} else {

    // Empty statement
    node->addChild(
        makeNonTerminal(
            NonTerminal::Empty
        )
    );

}

return node;
}

NodePtr Parser::parseExpression() {
    NodePtr node =
        makeNonTerminal(
            NonTerminal::Expression
        );

    // Parsing ekspresi matematika/logika
    node->addChild(
        parseSimpleExpression()
    );

    if (isRelationalOp()) {

        node->addChild(
            parseRelationalOperator()
        );

        node->addChild(

```

```

        parseSimpleExpression()
    );
}

return node;
}

NodePtr Parser::parseFactor() {
    NodePtr node =
        makeNonTerminal(
            NonTerminal::Factor
        );

    // Parsing factor expression
    if (check("LPARENT")) {

        node->addChild(termNode());

        node->addChild(
            parseExpression()
        );

        node->addChild(
            expectNode("RPARENT")
        );

    } else if (check("IDENT")) {

        node->addChild(
            parseVariable()
        );

    } else {

        node->addChild(
            termNode()
        );

    }

    return node;
}

NodePtr Parser::parseVariable() {
    NodePtr node =
        makeNonTerminal(
            NonTerminal::Variable
        );

    // Parsing variable dan selector
    node->addChild(
        expectNode("IDENT")
    );

    while (checkAny({
        "LBRACK",
        "PERIOD"
    }

```

```

    ))) {

        node->addChild(
            parseSelector()
        );
    }

    return node;
}

void printTree(const Node* root,
               std::ostream& out) {

    // Print parse tree hasil parser
    if (root == nullptr) {
        out << "<empty tree>\n";
        return;
    }

    root->print(out);
}

```

/*

Fungsi parser lain yang digunakan:

- parseIdentifierList()
Parsing daftar identifier
- parseBlock()
Parsing block program
- parseDeclarationPart()
Parsing deklarasi program
- parseConstDeclaration()
Parsing konstanta
- parseTypeDeclaration()
Parsing tipe data
- parseVarDeclaration()
Parsing variabel
- parseArrayType()
Parsing array
- parseRecordType()
Parsing record
- parseCompoundStatement()
Parsing begin-end
- parseIfStatement()
Parsing if statement
- parseCaseStatement()

Parsing case statement

- `parseRepeatStatement()`
Parsing repeat-until
 - `parseWhileStatement()`
Parsing while loop
 - `parseForStatement()`
Parsing for loop
 - `parseParameterList()`
Parsing parameter function/procedure
 - `parseTerm()`, `parseFactor()`
Parsing operasi matematika
 - `parseSelector()`
Parsing array index / field record
 - `parseConstant()`
Parsing konstanta
 - `isRelationalOp()`
Mengecek operator relasional
 - `isAdditiveOp()`
Mengecek operator penjumlahan
 - `isMultiplicativeOp()`
Mengecek operator perkalian
- */

Penjelasan

Kelas Parser digunakan untuk melakukan proses parsing terhadap token hasil lexer menggunakan metode *top down parsing*. Parser akan membaca token satu per satu berdasarkan grammar bahasa Arion yang telah ditentukan dan membangun struktur parse tree menggunakan node non-terminal maupun terminal. Class `ParseException` digunakan untuk menangani kesalahan sintaks selama parsing berlangsung. Error yang dihasilkan mencakup pesan kesalahan beserta posisi baris dan kolom token agar mempermudah debugging program.

Fungsi dasar seperti `current()`, `lookAhead()`, `consume()`, dan `expect()` berfungsi untuk mengelola proses pembacaan token. Fungsi `expect()` memiliki peran penting untuk memastikan token yang dibaca sesuai dengan grammar, sedangkan `consume()` digunakan untuk mengambil token dan memajukan posisi parser. Fungsi `parse()` merupakan entry point parser yang akan memulai parsing dari grammar paling atas yaitu `parseProgram()`. Selanjutnya parser akan memanggil fungsi parsing lain secara rekursif seperti `parseDeclarationPart()`, `parseStatement()`, `parseExpression()`, dan `parseVariable()` sesuai struktur grammar program. Pada bagian statement, parser mampu mengenali berbagai bentuk statement seperti assignment, procedure/function call, percabangan if, perulangan while, repeat, for, serta compound statement begin-end. Pada bagian expression, parser memproses ekspresi matematika dan logika menggunakan hirarki recursive descent mulai dari `Expression`, `SimpleExpression`, `Term`, hingga `Factor` sehingga prioritas operator tetap terjaga.

Selama parsing berlangsung, setiap hasil parsing akan disimpan ke dalam parse tree menggunakan helper function seperti `makeNonTerminal()` dan `makeTerminal()`. Parse tree tersebut kemudian dapat dicetak menggunakan fungsi `printTree()` untuk menampilkan struktur sintaks program secara hierarkis.

BAB III PENGUJIAN

3.1. Pengujian Pertama

Input (input1.txt)	Output (parse_tree1.txt)
<pre> MAR-Tubes-IF2224-2026 > test > milestone2 > input > input1.txt 1 program HelloWorld; 2 var 3 a, b: integer; 4 begin 5 a := 5; 6 b := a + 10; 7 writeln('Result = ', b); 8 end. </pre>	<pre> MAR-Tubes-IF2224-2026 > test > milestone2 > output > parse_tree1.txt 1 <program> 2 <program-header> 3 programsy (program) 4 ident (HelloWorld) 5 semicolon (;) 6 <declaration-part> 7 <var-declaration> </pre>
Output (token1.txt)	
<pre> MAR-Tubes-IF2224-2026 > test > milestone2 > output > token1.txt 1 programsy 2 ident (HelloWorld) 3 semicolon 4 varsy 5 ident (a) 6 comma 7 ident (b) </pre> programsy ident (HelloWorld) semicolon varsy ident (a) comma ident (b) colon ident (integer) semicolon beginsy ident (a) becomes intcon (5) semicolon ident (b) becomes ident (a) plus intcon (10) semicolon ident (writeln) lparent string ('Result = ') comma ident (b)	<pre> <program> <program-header> programsy (program) ident (HelloWorld) semicolon (;) <declaration-part> <var-declaration> varsy (var) <identifier-list> ident (a) comma (,) ident (b) colon (:) <type> ident (integer) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (a) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (5) semicolon (;) <statement> <assignment-statement> <variable> ident (b) becomes (:=) <expression> </pre>

rparent semicolon endsy period	<simple-expression> <term> <factor> <variable> ident (a) <additive-operator> plus (+) <term> <factor> intcon (10) semicolon (;) <statement> <procedure/function-call> ident (writeln) lparent (<parameter-list> <expression> <simple-expression> <term> <factor> string ('Result = ' comma (, <expression> <simple-expression> <term> <factor> <variable> ident (b) rparent ()) semicolon (;) <statement> <empty> endsy (end) period (.)
Keterangan	
Pengujian pertama bertujuan untuk memastikan parser dapat mengenali struktur program Arion paling dasar. Program ini memuat header program, deklarasi variabel, assignment statement, ekspresi aritmatika sederhana, dan pemanggilan prosedur writeln. Hasil token menunjukkan bahwa lexer berhasil mengklasifikasikan setiap komponen seperti programsy, ident, becomes, plus, dan string. Parse tree yang dihasilkan juga menunjukkan susunan program sesuai grammar, mulai dari <program-header>, <declaration-part>, hingga <compound-statement>.	

3.2. Pengujian Kedua

Input (input2.txt)	Output (parse_tree2.txt)
--------------------	--------------------------

```
MAR-Tubes-IF2224-2026 > test > milestone2 > input > 𐀀 input2.txt
1  program ConstAndType;
2  const
3    MAX == 100;
4    PI == 3.14;
5    INITIAL == 'A';
6    GREETING == 'Hello';
7  type
8    SmallRange == 1..10;
9    Season == (Spring, Summer, Fall, Winter);
10 var
11  x: integer;
12  ch: char;
13  begin
14    x := MAX;
15    ch := INITIAL;
16    writeln(x);
17  end.
```

Output (token2.txt)

```
MAR-Tubes-IF2224-2026 > test > milestone2 > output > 𐀀 token2.txt
1  programsy
2  ident (ConstAndType)
3  semicolon
4  constsy
5  ident (MAX)
6  eql
7  intcon (100)
```

programsy
ident (ConstAndType)
semicolon
constsy
ident (MAX)
eql
intcon (100)
semicolon
ident (PI)
eql
realcon (3.14)
semicolon
ident (INITIAL)
eql
charcon ('A')
semicolon
ident (GREETING)
eql
string ('Hello')
semicolon
typesy
ident (SmallRange)
eql
intcon (1)
period
period
intcon (10)
semicolon
ident (Season)
eql
lparent
ident (Spring)

```
MAR-Tubes-IF2224-2026 > test > milestone2 > output > 𐀀 parse_tree2.txt
1  <program>
2    <program-header>
3      programsy (program)
4        ident (ConstAndType)
5        semicolon (;)
6    <declaration-part>
7      <const-declaration>
8        constsy (const)
9        <const-definition>
```

<program>
<program-header>
 programsy (program)
 ident (ConstAndType)
 semicolon (;)
<declaration-part>
 <const-declaration>
 constsy (const)
 <const-definition>
 ident (MAX)
 eql (==)
 <constant>
 intcon (100)
 semicolon (;)
 <const-definition>
 ident (PI)
 eql (==)
 <constant>
 realcon (3.14)
 semicolon (;)
 <const-definition>
 ident (INITIAL)
 eql (==)
 <constant>
 charcon ('A')
 semicolon (;)
 <const-definition>
 ident (GREETING)
 eql (==)
 <constant>
 string ('Hello')
 semicolon (;)
 <type-declaration>
 typesy (type)
 <type-definition>
 ident (SmallRange)
 eql (==)
 <type>
 <range>
 <constant>
 intcon (1)
 period (.)
 period (.)

comma ident (Summer) comma ident (Fall) comma ident (Winter) rparent semicolon varsy ident (x) colon ident (integer) semicolon ident (ch) colon ident (char) semicolon beginsy ident (x) becomes ident (MAX) semicolon ident (ch) becomes ident (INITIAL) semicolon ident (writeln) lparent ident (x) rparent semicolon endsy period	<constant> intcon (10) semicolon (;) <type-definition> ident (Season) eql (==) <type> <enumerated> lparent () <identifier-list> ident (Spring) comma (,) ident (Summer) comma (,) ident (Fall) comma (,) ident (Winter) rparent ()) semicolon (;) <var-declaration> varsy (var) <identifier-list> ident (x) colon (:) <type> ident (integer) semicolon (;) <identifier-list> ident (ch) colon (:) <type> ident (char) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (x) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (MAX) semicolon (;) <statement> <assignment-statement>
--	--

	<pre> <variable> ident (ch) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (INITIAL) semicolon (;) <statement> <procedure/function-call> ident (writeln) lparent (()) <parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (x) rparent ()) semicolon (;) <statement> <empty> endsy (end) period (.) </pre>
Keterangan	
Pengujian kedua digunakan untuk memeriksa kemampuan parser dalam menangani deklarasi konstanta dan deklarasi tipe. Kasus ini mencakup konstanta integer, real, karakter, string, tipe range, serta tipe enumerasi. Output token menunjukkan bahwa simbol ==, tanda kurung, koma, dan konstanta berhasil dikenali dengan tipe token yang tepat. Parse tree yang terbentuk memperlihatkan bahwa parser dapat membangun node <const-declaration>, <type-declaration>, <range>, dan <enumerated> secara benar.	

3.3. Pengujian Ketiga

Input (input3.txt)	Output (parse_tree3.txt)
--------------------	--------------------------

```

MAR-Tubes-IF2224-2026 > test > milestone2 > input > E input3.txt
1  program ArrayAndRecord;
2  type
3    Matrix == array[1..3] of integer;
4    Person == record
5      name: string;
6      age: integer;
7      score: real;
8    end;
9  var
10   arr: array[1..5] of integer;
11   i: integer;
12   p: Person;
13 begin
14   arr[1] := 10;
15   arr[2] := 20;
16   arr[3] := arr[1] + arr[2];
17   p.age := 20;
18   p.score := 3.14;
19   for i := 1 to 5 do
20     writeln(arr[i]);
21 end.

```

Output (token3.txt)

```

MAR-Tubes-IF2224-2026 > test > milestone2 > output > E token3.txt
1  programsy
2  ident (ArrayAndRecord)
3  semicolon
4  typesy
5  ident (Matrix)
6  eql
7  arraysy
8  lbrack
9  intcon (1)

```

programsy
 ident (ArrayAndRecord)
 semicolon
 typesy
 ident (Matrix)
 eql
 arraysy
 lbrack
 intcon (1)
 period
 period
 intcon (3)
 rbrack
 ofsy
 ident (integer)
 semicolon
 ident (Person)
 eql
 recordsy
 ident (name)
 colon
 ident (string)
 semicolon
 ident (age)
 colon
 ident (integer)
 semicolon
 ident (score)
 colon
 ident (real)

```

MAR-Tubes-IF2224-2026 > test > milestone2 > output > E parse_tree3.txt
1  <program>
2  <program-header>
3  <programsy (program)>
4  <ident (ArrayAndRecord)>
5  <semicolon (;)>
6  <declaration-part>
7  <type-declaration>
8  <typesy (type)>
9  <type-definition>
10 <ident (Matrix)>
11 <eql (==)>
12 <type>

```

<program>
 <program-header>
 programsy (program)
 ident (ArrayAndRecord)
 semicolon (;)
 <declaration-part>
 <type-declaration>
 typesy (type)
 <type-definition>
 ident (Matrix)
 eql (==)
 <type>
 <array-type>
 arraysy (array)
 lbrack ([)
 <range>
 <constant>
 intcon (1)
 period (.)
 period (.)
 <constant>
 intcon (3)
 rbrack (])
 ofsy (of)
 <type>
 ident (integer)
 semicolon (;)
 <type-definition>
 ident (Person)
 eql (==)
 <type>
 <record-type>
 recordsy (record)
 <field-list>
 <field-part>
 <identifier-list>
 ident (name)
 colon (:)
 <type>
 ident (string)
 semicolon (;)
 <field-part>

endsy	<identifier-list>
semicolon	ident (age)
varsy	colon (:)
ident (arr)	<type>
colon	ident (integer)
arraysy	semicolon (;)
lbrack	<field-part>
intcon (1)	<identifier-list>
period	ident (score)
period	colon (:)
intcon (5)	<type>
rbrack	ident (real)
ofsy	endsy (end)
ident (integer)	semicolon (;)
semicolon	<var-declaration>
ident (i)	varsy (var)
colon	<identifier-list>
ident (integer)	ident (arr)
semicolon	colon (:)
ident (p)	<type>
colon	<array-type>
ident (Person)	arraysy (array)
semicolon	lbrack (l)
beginsy	<range>
ident (arr)	<constant>
lbrack	intcon (1)
intcon (1)	period (.)
rbrack	period (.)
becomes	<constant>
intcon (10)	intcon (5)
semicolon	rbrack (r)
ident (arr)	ofsy (of)
lbrack	<type>
intcon (2)	ident (integer)
rbrack	semicolon (;)
becomes	<identifier-list>
intcon (20)	ident (i)
semicolon	colon (:)
ident (arr)	<type>
lbrack	ident (integer)
intcon (3)	semicolon (;)
rbrack	<identifier-list>
becomes	ident (p)
ident (arr)	colon (:)
lbrack	<type>
intcon (1)	ident (Person)
rbrack	semicolon (;)
plus	<compound-statement>
ident (arr)	beginsy (begin)
lbrack	<statement-list>

intcon (2) rbrack semicolon ident (p) period ident (age) becomes intcon (20) semicolon ident (p) period ident (score) becomes realcon (3.14) semicolon forsy ident (i) becomes intcon (1) tosy intcon (5) dosy ident (writeln) lparent ident (arr) lbrack ident (i) rbrack rparent semicolon endsy period	<statement> <assignment-statement> <variable> ident (arr) <selector> lbrack (] <index-list> <expression> <simple-expression> <term> <factor> intcon (1) rbrack (] becomes (:=) <expression> <simple-expression> <term> <factor> intcon (10) semicolon (;) <statement> <assignment-statement> <variable> ident (arr) <selector> lbrack (] <index-list> <expression> <simple-expression> <term> <factor> intcon (2) rbrack (] becomes (:=) <expression> <simple-expression> <term> <factor> intcon (20) semicolon (;) <statement> <assignment-statement> <variable> ident (arr) <selector> lbrack (] <index-list> <expression> <simple-expression> <term>
--	--

	<factor> intcon (3) rbrack (]) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (arr) <selector> lbrack ([) <index-list> <expression> <simple-expression> <term> <factor> intcon (1) rbrack (]) <additive-operator> plus (+) <term> <factor> <variable> ident (arr) <selector> lbrack ([) <index-list> <expression> <simple-expression> <term> <factor> intcon (2) rbrack (]) semicolon (;) <statement> <assignment-statement> <variable> ident (p) <selector> period (.) ident (age) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (20) semicolon (;) <statement>
--	--

	<assignment-statement> <variable> ident (p) <selector> period (.) ident (score) becomes (:=) <expression> <simple-expression> <term> <factor> realcon (3.14) semicolon (;) <statement> <for-statement> forsy (for) ident (i) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (1) tosy (to) <expression> <simple-expression> <term> <factor> intcon (5) dosy (do) <statement> <procedure/function-call> ident (writeln) lparent (<parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (arr) <selector> lbrack ([<index-list> <expression> <simple-expression> <term> <factor> <variable> ident (i)
--	---

	rbrack (]) rparent ()) semicolon (;) <statement> <empty> endsy (end) period (.)
Keterangan	
Pengujian ketiga berfokus pada pemrosesan tipe data kompleks, yaitu array dan record. Program ini menguji deklarasi array, deklarasi record, pengaksesan elemen array, pengaksesan field record, dan perulangan for. Hasil token menunjukkan bahwa lexer mampu mengenali tanda kurung siku, operator titik, keyword array, record, of, dan token lain yang dibutuhkan. Parse tree yang dihasilkan membuktikan bahwa parser dapat membentuk struktur <array-type>, <record-type>, <variable>, serta selector untuk akses array dan field.	

3.4. Pengujian Keempat

Input (input4.txt)	Output (parse_tree4.txt)
<pre> MAR-Tubes-IF2224-2026 > test > milestone2 > input >  input4.txt 1 program BranchingStatements; 2 var 3 x, y, z: integer; 4 begin 5 x := 10; 6 y := 20; 7 if x == y then 8 writeln('sama') 9 else begin 10 if x > y then 11 z := x 12 else 13 z := y; 14 writeln(z) 15 end; 16 case x of 17 1: writeln('satu'); 18 2, 3: writeln('dua atau tiga'); 19 10: writeln('sepuluh') 20 end 21 end. </pre>	<pre> MAR-Tubes-IF2224-2026 > test > milestone2 > output >  parse_tree4.txt 1 <program> 2 <program-header> 3 programsy (program) 4 ident (BranchingStatements) 5 semicolon (;) 6 <declaration-part> 7 <var-declaration> 8 varsy (var) 9 <identifier-list> 10 ident (x) 11 comma (,) 12 ident (y) 13 comma (,) 14 ident (z) </pre>
Output (token4.txt)	
<pre> MAR-Tubes-IF2224-2026 > test > milestone2 > output >  token4.txt 1 programsy 2 ident (BranchingStatements) 3 semicolon 4 varsy 5 ident (x) 6 comma 7 ident (y) 8 comma 9 ident (z) 10 colon 11 ident (integer) </pre>	<pre> <program> <program-header> programsy (program) ident (BranchingStatements) semicolon (;) <declaration-part> <var-declaration> varsy (var) <identifier-list> ident (x) comma (,) ident (y) comma (,) ident (z) colon (:) <type> ident (integer) semicolon (;) </pre>

varsy ident (x) comma ident (y) comma ident (z) colon ident (integer) semicolon beginsy ident (x) becomes intcon (10) semicolon ident (y) becomes intcon (20) semicolon ifsy ident (x) eql ident (y) thensy ident (writeln) lparent string ('sama') rparent elsesy beginsy ifsy ident (x) gtr ident (y) thensy ident (z) becomes ident (x) elsesy ident (z) becomes ident (y) semicolon ident (writeln) lparent ident (z) rparent endsy semicolon casesy ident (x)	<compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (x) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (10) semicolon (;) <statement> <assignment-statement> <variable> ident (y) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (20) semicolon (;) <statement> <if-statement> ifsy (if) <expression> <simple-expression> <term> <factor> <variable> ident (x) <relational-operator> eql (==) <simple-expression> <term> <factor> <variable> ident (y) thensy (then) <statement> <procedure/function-call> ident (writeln) lparent () <parameter-list> <expression> <simple-expression> <term>
---	---

ofsy intcon (1) colon ident (writeln) lparent string ('satu') rparent semicolon intcon (2) comma intcon (3) colon ident (writeln) lparent string ('dua atau tiga') rparent semicolon intcon (10) colon ident (writeln) lparent string ('sepuluh') rparent endsy endsy period	<factor> string ('sama') rparent ()) elsesy (else) <statement> <compound-statement> beginsy (begin) <statement-list> <statement> <if-statement> ifsy (if) <expression> <simple-expression> <term> <factor> <variable> ident (x) <relational-operator> gtr (>) <simple-expression> <term> <factor> <variable> ident (y) thensy (then) <statement> <assignment-statement> <variable> ident (z) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (x) elsesy (else) <statement> <assignment-statement> <variable> ident (z) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (y) semicolon (;) <statement>
---	---

	<pre> <procedure/function-call> ident (writeln) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (z) rparent () endsy (end) semicolon (;) <statement> <case-statement> casesy (case) <expression> <simple-expression> <term> <factor> <variable> ident (x) ofsy (of) <case-block> <constant> intcon (1) colon (:) <statement> <procedure/function-call> ident (writeln) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> string ('satu') rparent () semicolon (;) <case-block> <constant> intcon (2) comma (,) <constant> intcon (3) colon (:) <statement> <procedure/function-call> ident (writeln) lparent () </pre>
--	---

	<pre> <parameter-list> <expression> <simple-expression> <term> <factor> string ('dua atau tiga') rparent ()) semicolon (;) <case-block> <constant> intcon (10) colon (:) <statement> <procedure/function-call> ident (writeln) lparent (()) <parameter-list> <expression> <simple-expression> <term> <factor> string ('sepuluh') rparent ()) endsy (end) endsy (end) period (.) </pre>
Keterangan	
Pengujian keempat digunakan untuk menguji struktur percabangan dalam bahasa Arion. Program ini memuat statement if-then-else, nested if, compound statement di dalam cabang else, serta case statement dengan beberapa pilihan nilai. Output token memperlihatkan bahwa operator relasional seperti == dan > berhasil dikenali bersama keyword percabangan seperti if, then, else, case, dan of. Parse tree yang dihasilkan menunjukkan bahwa parser mampu membangun node <if-statement> dan <case-statement> secara bertingkat sesuai struktur input.	

3.5. Pengujian Kelima

Input (input5.txt)	Output (parse_tree5.txt)
---------------------------	---------------------------------


```

MAR-Tubes-IF2224-2026 > test > milestone2 > input >  input5.txt
1  program LoopStatements;
2  var
3    i, sum, factorial: integer;
4  begin
5    sum := 0;
6    i := 1;
7    while i <= 10 do begin
8      sum := sum + i;
9      i := i + 1
10   end;
11   factorial := 1;
12   i := 5;
13   repeat
14     factorial := factorial * i;
15     i := i - 1
16   until i == 0;
17   for i := 10 downto 1 do
18     writeln(i);
19     writeln(sum);
20     writeln(factorial)
21   end.

```

Output (token5.txt)

```

MAR-Tubes-IF2224-2026 > test > milestone2 > output >  token5.txt
1  programsy
2  ident (LoopStatements)
3  semicolon
4  varsy
5  ident (i)
6  comma
7  ident (sum)
8  comma
9  ident (factorial)
10 colon
11 ident (integer)
12 semicolon
13 beginsy
14 ident (sum)
15 becomes

```

```

programsy
ident (LoopStatements)
semicolon
varsy
ident (i)
comma
ident (sum)
comma
ident (factorial)
colon
ident (integer)
semicolon
beginsy
ident (sum)
becomes
intcon (0)
semicolon
ident (i)
becomes
intcon (1)
semicolon
whilesy
ident (i)
leq
intcon (10)
dosy
beginsy
ident (sum)

```

```

MAR-Tubes-IF2224-2026 > test > milestone2 > output >  parse_tree5.txt
1  <program>
2  <program-header>
3    programsy (program)
4    ident (LoopStatements)
5    semicolon (;)
6  <declaration-part>
7    <var-declaration>
8      varsy (var)
9      <identifier-list>
10        ident (i)
11        comma (,)
12        ident (sum)
13        comma (,)
14        ident (factorial)

```

```

<program>
<program-header>
  programsy (program)
  ident (LoopStatements)
  semicolon (;)
<declaration-part>
  <var-declaration>
    varsy (var)
    <identifier-list>
      ident (i)
      comma (,)
      ident (sum)
      comma (,)
      ident (factorial)
    colon (:)
  <type>
    ident (integer)
    semicolon (;)
  <compound-statement>
    beginsy (begin)
    <statement-list>
      <statement>
        <assignment-statement>
          <variable>
            ident (sum)
          becomes (:=)
          <expression>
            <simple-expression>
              <term>
                <factor>
                  intcon (0)
            semicolon (;)
          <statement>
            <assignment-statement>
              <variable>
                ident (i)
              becomes (:=)
              <expression>
                <simple-expression>
                  <term>
                    <factor>

```

becomes ident (sum) plus ident (i) semicolon ident (i) becomes ident (i) plus intcon (1) endsy semicolon ident (factorial) becomes intcon (1) semicolon ident (i) becomes intcon (5) semicolon repeatsy ident (factorial) becomes ident (factorial) times ident (i) semicolon ident (i) becomes ident (i) minus intcon (1) untilsy ident (i) eql intcon (0) semicolon forsy ident (i) becomes intcon (10) downtosy intcon (1) dosy ident (writeln) lparent ident (i) rparent semicolon ident (writeln)	intcon (1) semicolon (;) <statement> <while-statement> whilesy (while) <expression> <simple-expression> <term> <factor> <variable> ident (i) <relational-operator> leq (<=) <simple-expression> <term> <factor> intcon (10) dosy (do) <statement> <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (sum) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (sum) <additive-operator> plus (+) <term> <factor> <variable> ident (i) semicolon (;) <statement> <assignment-statement> <variable> ident (i) becomes (:=) <expression> <simple-expression> <term> <factor> <variable>
---	--

lparent	ident (i)
ident (sum)	<additive-operator>
rparent	plus (+)
semicolon	<term>
ident (writeln)	<factor>
lparent	intcon (1)
ident (factorial)	endsy (end)
rparent	semicolon (;)
endsy	<statement>
period	<assignment-statement>
	<variable>
	ident (factorial)
	becomes (:=)
	<expression>
	<simple-expression>
	<term>
	<factor>
	intcon (1)
	semicolon (;)
	<statement>
	<assignment-statement>
	<variable>
	ident (i)
	becomes (:=)
	<expression>
	<simple-expression>
	<term>
	<factor>
	intcon (5)
	semicolon (;)
	<statement>
	<repeat-statement>
	repeatsy (repeat)
	<statement-list>
	<statement>
	<assignment-statement>
	<variable>
	ident (factorial)
	becomes (:=)
	<expression>
	<simple-expression>
	<term>
	<factor>
	<variable>
	ident (factorial)
	<multiplicative-operator>
	times (*)
	<factor>
	<variable>
	ident (i)

	semicolon (;) <statement> <assignment-statement> <variable> ident (i) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (i) <additive-operator> minus (-) <term> <factor> intcon (1) untilsy (until) <expression> <simple-expression> <term> <factor> <variable> ident (i) <relational-operator> eql (==) <simple-expression> <term> <factor> intcon (0) semicolon (;) <statement> <for-statement> forsy (for) ident (i) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (10) downtosy (downto) <expression> <simple-expression> <term> <factor> intcon (1) dosy (do) <statement> <procedure/function-call>
--	--

	<pre> ident (writeln) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (i) rparent () semicolon (;) <statement> <procedure/function-call> ident (writeln) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (sum) rparent () semicolon (;) <statement> <procedure/function-call> ident (writeln) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (factorial) rparent () endsy (end) period (.) </pre>
Keterangan Pengujian kelima bertujuan untuk memastikan parser dapat menangani berbagai bentuk perulangan. Program ini memuat while-do, repeat-until, dan for-downto dalam satu source code. Hasil token menunjukkan bahwa keyword perulangan serta operator seperti <=, ==, *, dan - diklasifikasikan dengan benar. Parse tree yang terbentuk memperlihatkan bahwa parser berhasil mengenali <while-statement>, <repeat-statement>, dan <for-statement> beserta statement yang berada di dalamnya.	

3.6. Pengujian Keenam

Input (input6.txt)	Output (parse_tree6.txt)
<pre> MAR-Tubes-IF2224-2026 > test > milestone2 > input > input6.txt 1 program ProcedureAndFunction; 2 procedure Greet(name: string); 3 begin 4 writeln('Hello, ', name) 5 end; 6 procedure Swap(a, b: integer); 7 var 8 temp: integer; 9 begin program ProcedureAndFunction; procedure Greet(name: string); begin writeln('Hello, ', name) end; procedure Swap(a, b: integer); var temp: integer; begin temp := a; a := b; b := temp end; function Max(a, b: integer): integer; begin if a > b then Max := a else Max := b end; function Factorial(n: integer): integer; begin if n == 0 then Factorial := 1 else Factorial := n * Factorial(n - 1) end; var x, y: integer; begin x := 5; y := 3; Greet('World'); Swap(x, y); writeln(Max(x, y)); writeln(Factorial(5)) end. </pre>	<pre> MAR-Tubes-IF2224-2026 > test > milestone2 > output > parse_tree6.txt 1 <program> 2 <program-header> 3 programsy (program) 4 ident (ProcedureAndFunction) 5 semicolon (;) 6 <declaration-part> 7 <subprogram-declaration-part> 8 <subprogram-declaration> 9 <procedure-declaration> 10 <procedure-heading> 11 proceduresy (procedure) 12 ident (Greet) 13 <formal-parameter-list> 14 lparent (()) 15 <parameter-group> 16 <identifier-list> <program> <program-header> programsy (program) ident (ProcedureAndFunction) semicolon (;) <declaration-part> <subprogram-declaration-part> <subprogram-declaration> <procedure-declaration> <procedure-heading> proceduresy (procedure) ident (Greet) <formal-parameter-list> lparent (()) <parameter-group> <identifier-list> ident (name) colon (:) ident (string) rparent (()) semicolon (;) <block> <declaration-part> <compound-statement> beginsy (begin) <statement-list> <statement> <procedure/function-call> ident (writeln) lparent (()) <parameter-list> <expression> <simple-expression> <term> <factor> string ('Hello, ') </pre>
Output (token6.txt)	

MAR-Tubes-IF2224-2026 > test > milestone2 > output > token6.txt

```
1  programsy
2  ident (ProcedureAndFunction)
3  semicolon
4  proceduressy
5  ident (Greet)
6  lparent
7  ident (name)
8  colon
9  ident (string)
10 rparent
11 semicolon
12 beginsy
13 ident (writeln)
```

programsy
ident (ProcedureAndFunction)
semicolon
proceduressy
ident (Greet)
lparent
ident (name)
colon
ident (string)
rparent
semicolon
beginsy
ident (writeln)
lparent
string ('Hello, ')
comma
ident (name)
rparent
endsy
semicolon
proceduressy
ident (Swap)
lparent
ident (a)
comma
ident (b)
colon
ident (integer)
rparent
semicolon
varsy
ident (temp)
colon
ident (integer)
semicolon
beginsy
ident (temp)
becomes
ident (a)
semicolon
ident (a)
becomes

comma (,)
<expression>
<simple-expression>
<term>
<factor>
<variable>
ident (name)
rparent ())
endsy (end)
semicolon (;)
<subprogram-declaration>
<procedure-declaration>
<procedure-heading>
proceduressy (procedure)
ident (Swap)
<formal-parameter-list>
lparent ()
<parameter-group>
<identifier-list>
ident (a)
comma (,)
ident (b)
colon (:)
ident (integer)
rparent ())
semicolon (;)
<block>
<declaration-part>
<var-declaration>
varsy (var)
<identifier-list>
ident (temp)
colon (:)
<type>
ident (integer)
semicolon (;)
<compound-statement>
beginsy (begin)
<statement-list>
<statement>
<assignment-statement>
<variable>
ident (temp)
becomes (:=)
<expression>
<simple-expression>
<term>
<factor>
<variable>
ident (a)

ident (b) semicolon ident (b) becomes ident (temp) endsy semicolon functionsy ident (Max) lparent ident (a) comma ident (b) colon ident (integer) rparent colon ident (integer) semicolon beginsy ifsy ident (a) gtr ident (b) thensy ident (Max) becomes ident (a) elsesy ident (Max) becomes ident (b) endsy semicolon functionsy ident (Factorial) lparent ident (n) colon ident (integer) rparent colon ident (integer) semicolon beginsy ifsy ident (n) eq intcon (0) thensy	semicolon (;) <statement> <assignment-statement> <variable> ident (a) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (b) semicolon (;) <statement> <assignment-statement> <variable> ident (b) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (temp) endsy (end) semicolon (;) <subprogram-declaration> <function-declaration> <function-heading> functionsy (function) ident (Max) <formal-parameter-list> lparent () <parameter-group> <identifier-list> ident (a) comma (,) ident (b) colon (:) ident (integer) rparent ()) colon (:) ident (integer) semicolon (;) <block> <declaration-part> <compound-statement> beginsy (begin) <statement-list> <statement>
--	--

ident (Factorial) becomes intcon (1) elsesy ident (Factorial) becomes ident (n) times ident (Factorial) lparent ident (n) minus intcon (1) rparent endsy semicolon varsy ident (x) comma ident (y) colon ident (integer) semicolon beginsy ident (x) becomes intcon (5) semicolon ident (y) becomes intcon (3) semicolon ident (Greet) lparent string ('World') rparent semicolon ident (Swap) lparent ident (x) comma ident (y) rparent semicolon ident (writeln) lparent ident (Max) lparent ident (x) comma	<if-statement> ifsy (if) <expression> <simple-expression> <term> <factor> <variable> ident (a) <relational-operator> gtr (>) <simple-expression> <term> <factor> <variable> ident (b) thensy (then) <statement> <assignment-statement> <variable> ident (Max) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (a) elsesy (else) <statement> <assignment-statement> <variable> ident (Max) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (b) endsy (end) semicolon (;) <subprogram-declaration> <function-declaration> <function-heading> functionsy (function) ident (Factorial) <formal-parameter-list> lparent () <parameter-group> <identifier-list>
---	--

ident (y) rparent rparent semicolon ident (writeln) lparent ident (Factorial) lparent intcon (5) rparent rparent endsy period	ident (n) colon (:) ident (integer) rparent ()) colon (:) ident (integer) semicolon (;) <block> <declaration-part> <compound-statement> beginsy (begin) <statement-list> <statement> <if-statement> ifsy (if) <expression> <simple-expression> <term> <factor> <variable> ident (n) <relational-operator> eql (==) <simple-expression> <term> <factor> intcon (0) thensy (then) <statement> <assignment-statement> <variable> ident (Factorial) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (1) elsesy (else) <statement> <assignment-statement> <variable> ident (Factorial) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (n)
--	---

	<multiplicative-operator> times (*) <factor> <procedure/function-call> ident (Factorial) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (n) <additive-operator> minus (-) <term> <factor> intcon (1) rparent () endsy (end) semicolon (;) <var-declaration> varsy (var) <identifier-list> ident (x) comma (,) ident (y) colon (:) <type> ident (integer) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (x) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (5) semicolon (;) <statement> <assignment-statement> <variable> ident (y) becomes (:=)
--	--

```

    <expression>
      <simple-expression>
        <term>
          <factor>
            intcon (3)
semicolon (;)
<statement>
  <procedure/function-call>
    ident (Greet)
    lparent ()
    <parameter-list>
      <expression>
        <simple-expression>
          <term>
            <factor>
              string ('World')
    rparent ()
semicolon (;)
<statement>
  <procedure/function-call>
    ident (Swap)
    lparent ()
    <parameter-list>
      <expression>
        <simple-expression>
          <term>
            <factor>
              <variable>
                ident (x)
      comma (,)
      <expression>
        <simple-expression>
          <term>
            <factor>
              <variable>
                ident (y)
    rparent ()
semicolon (;)
<statement>
  <procedure/function-call>
    ident (writeln)
    lparent ()
    <parameter-list>
      <expression>
        <simple-expression>
          <term>
            <factor>
              <procedure/function-call>
                ident (Max)
            lparent ()

```

	<pre> <parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (x) comma (,) <expression> <simple-expression> <term> <factor> <variable> ident (y) rparent ()) rparent () semicolon (;) <statement> <procedure/function-call> ident (writeln) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> <procedure/function-call> ident (Factorial) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> intcon (5) rparent ()) rparent () rparent () endsy (end) period (.) </pre>
--	--

Keterangan

Pengujian keenam difokuskan pada deklarasi dan pemanggilan subprogram. Program ini mencakup deklarasi prosedur, deklarasi fungsi, parameter formal, variabel lokal, pemanggilan prosedur, pemanggilan fungsi, dan rekursi pada fungsi Factorial. Output token menunjukkan bahwa lexer mampu mengenali token seperti proceduressy, functionsy, lparent, rparent, colon, dan token ekspresi lain yang digunakan dalam subprogram. Parse tree membuktikan bahwa parser dapat membentuk node <procedure-declaration>, <function-declaration>, <formal-parameter-list>, dan <procedure/function-call> dengan struktur yang sesuai.

3.7. Pengujian Ketujuh

Input (input7.txt)	Output (parse_tree7.txt)
<pre> MAR-Tubes-IF2224-2026 > test > milestone2 > input > input7.txt 1 program Expressions; 2 var 3 a, b, c: integer; 4 x, y: real; 5 p, q, r: boolean; 6 begin 7 a := 10; 8 b := 3; 9 c := a + b * 2 - 1; 10 c := a div b; 11 c := a mod b; 12 x := 10.0 / 3.0; 13 c := -a + b; 14 p := a == b; 15 q := a <> b; 16 r := (a > 0) AND (b > 0); 17 r := (a > 0) OR (b < 0); 18 r := NOT (a == b); 19 if (a > b) AND NOT (a == 0) then 20 writeln(a) 21 end. </pre>	<pre> MAR-Tubes-IF2224-2026 > test > milestone2 > output > parse_tree7.txt 1 <program> 2 <program-header> 3 programsy (program) 4 ident (Expressions) 5 semicolon (;) 6 <declaration-part> 7 <var-declaration> 8 varsy (var) 9 <identifier-list> 10 ident (a) 11 comma (,) 12 ident (b) 13 comma (,) 14 ident (c) 15 colon (:) 16 <type> 17 ident (integer) </pre>
Output (token7.txt)	
<pre> MAR-Tubes-IF2224-2026 > test > milestone2 > output > token7.txt 1 programsy 2 ident (Expressions) 3 semicolon 4 varsy 5 ident (a) 6 comma 7 ident (b) 8 comma 9 ident (c) 10 colon 11 ident (integer) 12 semicolon 13 ident (x) </pre> programsy ident (Expressions) semicolon varsy ident (a) comma ident (b) comma ident (c) colon ident (integer) semicolon ident (x) comma ident (y) colon ident (real) semicolon ident (p) comma ident (q) comma	<pre> <program> <program-header> programsy (program) ident (Expressions) semicolon (;) <declaration-part> <var-declaration> varsy (var) <identifier-list> ident (a) comma (,) ident (b) comma (,) ident (c) colon (:) <type> ident (integer) semicolon (;) <identifier-list> ident (p) comma (,) ident (q) comma (,) ident (r) colon (:) <type> ident (boolean) semicolon (;) </pre>

ident (r) colon ident (boolean) semicolon beginsy ident (a) becomes intcon (10) semicolon ident (b) becomes intcon (3) semicolon ident (c) becomes ident (a) plus ident (b) times intcon (2) minus intcon (1) semicolon ident (c) becomes ident (a) idiv ident (b) semicolon ident (c) becomes ident (a) imod ident (b) semicolon ident (x) becomes realcon (10.0) rdiv realcon (3.0) semicolon ident (c) becomes minus ident (a) plus ident (b) semicolon ident (p) becomes	<compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (a) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (10) semicolon (;) <statement> <assignment-statement> <variable> ident (b) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (3) semicolon (;) <statement> <assignment-statement> <variable> ident (c) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (a) <additive-operator> plus (+) <term> <factor> <variable> ident (b) <multiplicative-operator> times (*) <factor> intcon (2) <additive-operator> minus (-) <term> <factor>
--	---

ident (a) eql ident (b) semicolon ident (q) becomes ident (a) neq ident (b) semicolon ident (r) becomes lparent ident (a) gtr intcon (0) rparent andsy lparent ident (b) gtr intcon (0) rparent semicolon ident (r) becomes lparent ident (a) gtr intcon (0) rparent orsy lparent ident (b) lss intcon (0) rparent semicolon ident (r) becomes notsy lparent ident (a) eql ident (b) rparent semicolon ifsy lparent ident (a)	intcon (1) semicolon () <statement> $\text{<assignment-statement>}$ <variable> ident (c) becomes (:=) <expression> $\text{<simple-expression>}$ <term> <factor> <variable> ident (a) $\text{<multiplicative-operator>}$ idiv (div) <factor> <variable> ident (b) semicolon () <statement> $\text{<assignment-statement>}$ <variable> ident (c) becomes (:=) <expression> $\text{<simple-expression>}$ <term> <factor> <variable> ident (a) $\text{<multiplicative-operator>}$ imod (mod) <factor> <variable> ident (b) semicolon () <statement> $\text{<assignment-statement>}$ <variable> ident (x) becomes (:=) <expression> $\text{<simple-expression>}$ <term> <factor> realcon (10.0) $\text{<multiplicative-operator>}$ rdiv (/) <factor> realcon (3.0)
---	--

gtr	semicolon (;)
ident (b)	<statement>
rparent	<assignment-statement>
andsy	<variable>
notsy	ident (c)
lparent	becomes (:=)
ident (a)	<expression>
eql	<simple-expression>
intcon (0)	minus (-)
rparent	<term>
thensy	<factor>
ident (writeln)	<variable>
lparent	ident (a)
ident (a)	<additive-operator>
rparent	plus (+)
endsy	<term>
period	<factor>
	<variable>
	ident (b)
	semicolon (;)
	<statement>
	<assignment-statement>
	<variable>
	ident (p)
	becomes (:=)
	<expression>
	<simple-expression>
	<term>
	<factor>
	<variable>
	ident (a)
	<relational-operator>
	eql (==)
	<simple-expression>
	<term>
	<factor>
	<variable>
	ident (b)
	semicolon (;)
	<statement>
	<assignment-statement>
	<variable>
	ident (q)
	becomes (:=)
	<expression>
	<simple-expression>
	<term>
	<factor>
	<variable>
	ident (a)

	<relational-operator> neq (< >) <simple-expression> <term> <factor> <variable> ident (b) semicolon (;) <statement> <assignment-statement> <variable> ident (r) becomes (:=) <expression> <simple-expression> <term> <factor> lparent (<expression> <simple-expression> <term> <factor> <variable> ident (a) <relational-operator> gtr (>) <simple-expression> <term> <factor> intcon (0) rparent ()) <multiplicative-operator> andsy (AND) <factor> lparent (<expression> <simple-expression> <term> <factor> <variable> ident (b) <relational-operator> gtr (>) <simple-expression> <term> <factor> intcon (0) rparent ()) semicolon (;) <statement>
--	--

	<pre> <assignment-statement> <variable> ident (r) becomes (:=) <expression> <simple-expression> <term> <factor> lparent (() <expression> <simple-expression> <term> <factor> <variable> ident (a) <relational-operator> gtr (>) <simple-expression> <term> <factor> intcon (0) rparent ()) <additive-operator> orsy (OR) <term> <factor> lparent (() <expression> <simple-expression> <term> <factor> <variable> ident (b) <relational-operator> lss (<) <simple-expression> <term> <factor> intcon (0) rparent ()) semicolon (;) <statement> <assignment-statement> <variable> ident (r) becomes (:=) <expression> <simple-expression> <term> <factor> </pre>
--	--

	notsy (NOT) <factor> lparent (() <expression> <simple-expression> <term> <factor> <variable> ident (a) <relational-operator> eql (==) <simple-expression> <term> <factor> <variable> ident (b) rparent ()) semicolon (;) <statement> <if-statement> ifsy (if) <expression> <simple-expression> <term> <factor> lparent (() <expression> <simple-expression> <term> <factor> <variable> ident (a) <relational-operator> gtr (>) <simple-expression> <term> <factor> <variable> ident (b) rparent ()) <multiplicative-operator> andsy (AND) <factor> notsy (NOT) <factor> lparent (() <expression> <simple-expression> <term> <factor>
--	---

	<pre> <variable> ident (a) <relational-operator> eql (==) <simple-expression> <term> <factor> intcon (0) rparent () thensy (then) <statement> <procedure/function-call> ident (writeln) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (a) rparent () endsy (end) period (.) </pre>
Keterangan	
Pengujian ketujuh digunakan untuk memvalidasi parsing ekspresi yang lebih kompleks. Program ini mencakup operasi aritmatika, pembagian integer, pembagian real, modulo, unary minus, operator relasional, serta operator logika AND, OR, dan NOT. Output token menunjukkan bahwa operator dengan prioritas berbeda berhasil dikenali sebagai token yang sesuai, seperti times, idiv, rdiv, imod, andsy, orsy, dan notsy. Parse tree menunjukkan bahwa parser berhasil memisahkan struktur <expression>, <simple-expression>, <term>, dan <factor> sehingga prioritas operator dapat direpresentasikan dengan jelas.	

3.8. Pengujian Kedelapan

Input (input8.txt)	Output (parse_tree8.txt)
---------------------------	---------------------------------

```

MAR-Tubes-IF2224-2026 > test > milestone2 > input > E input8.txt
1  program CommentsAndConstants;
2  { Ini adalah program untuk menguji komentar dan konstanta }
3  const
4      MAXSIZE == 50;
5      MINVAL == -10;
6      LABEL == 'Data';
7  var
8      i: integer;
9      ch: char;
10     s: string;
11 begin
12     { komentar di dalam begin }
13     i := MAXSIZE;
14     (* ini juga komentar
15      * multi baris *)
16     ch := 'Z';
17     s := 'Hello World';
18     i := MINVAL + 5;
19     writeln(i);
20     writeln(ch);
21     writeln(s);
22 end.

```

Output (token8.txt)

```

MAR-Tubes-IF2224-2026 > test > milestone2 > output > E token8.txt
1  programsy
2  ident (CommentsAndConstants)
3  semicolon
4  comment ({ Ini adalah program untuk menguji komentar dan konstanta })
5  constsy
6  ident (MAXSIZE)
7  eql
8  intcon (50)
9  semicolon
10 ident (MINVAL)
11 eql
12 minus

```

programsy
 ident (CommentsAndConstants)
 semicolon
 comment ({ Ini adalah program untuk menguji
 komentar dan konstanta })
 constsy
 ident (MAXSIZE)
 eql
 intcon (50)
 semicolon
 ident (MINVAL)
 eql
 minus
 intcon (10)
 semicolon
 ident (LABEL)
 eql
 string ('Data')
 semicolon
 varsy
 ident (i)
 colon
 ident (integer)
 semicolon
 ident (ch)
 colon
 ident (char)
 semicolon
 ident (s)
 colon

```

MAR-Tubes-IF2224-2026 > test > milestone2 > output > E parse_tree8.txt
1  <program>
2  <program-header>
3  programsy (program)
4  ident (CommentsAndConstants)
5  semicolon (;)
6  <declaration-part>
7  <const-declaration>
8  constsy (const)
9  <const-definition>
10 ident (MAXSIZE)
11 eql (==)
12 <constant>
13 intcon (50)
14 semicolon (;)

```

<program>
 <program-header>
 programsy (program)
 ident (CommentsAndConstants)
 semicolon (;)
 <declaration-part>
 <const-declaration>
 constsy (const)
 <const-definition>
 ident (MAXSIZE)
 eql (==)
 <constant>
 intcon (50)
 semicolon (;)
 <const-definition>
 ident (MINVAL)
 eql (==)
 <constant>
 minus (-)
 intcon (10)
 semicolon (;)
 <const-definition>
 ident (LABEL)
 eql (==)
 <constant>
 string ('Data')
 semicolon (;)
 <var-declaration>
 varsy (var)
 <identifier-list>
 ident (i)
 colon (:)
 <type>
 ident (integer)
 semicolon (;)
 <identifier-list>
 ident (ch)
 colon (:)
 <type>
 ident (char)
 semicolon (;)
 <identifier-list>

ident (string) semicolon beginsy comment ({ komentar di dalam begin }) ident (i) becomes ident (MAXSIZE) semicolon comment ((* ini juga komentar multi baris *)) ident (ch) becomes charcon ('Z') semicolon ident (s) becomes string ('Hello World') semicolon ident (i) becomes ident (MINVAL) plus intcon (5) semicolon ident (writeln) lparent ident (i) rparent semicolon ident (writeln) lparent ident (ch) rparent semicolon ident (writeln) lparent ident (s) rparent endsy period	ident (s) colon (:) <type> ident (string) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (i) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (MAXSIZE) semicolon (;) <statement> <assignment-statement> <variable> ident (ch) becomes (:=) <expression> <simple-expression> <term> <factor> charcon ('Z') semicolon (;) <statement> <assignment-statement> <variable> ident (s) becomes (:=) <expression> <simple-expression> <term> <factor> string ('Hello World') semicolon (;) <statement> <assignment-statement> <variable> ident (i) becomes (:=) <expression> <simple-expression> <term>
---	---

```

    <factor>
    <variable>
    ident (MINVAL)
    <additive-operator>
    plus (+)
    <term>
    <factor>
    intcon (5)
semicolon (;)
<statement>
    <procedure/function-call>
    ident (writeln)
    lparent (())
    <parameter-list>
    <expression>
    <simple-expression>
    <term>
    <factor>
    <variable>
    ident (i)
    rparent ())
semicolon (;)
<statement>
    <procedure/function-call>
    ident (writeln)
    lparent (())
    <parameter-list>
    <expression>
    <simple-expression>
    <term>
    <factor>
    <variable>
    ident (ch)
    rparent ())
semicolon (;)
<statement>
    <procedure/function-call>
    ident (writeln)
    lparent (())
    <parameter-list>
    <expression>
    <simple-expression>
    <term>
    <factor>
    <variable>
    ident (s)
    rparent ())
endsy (end)
period (.)

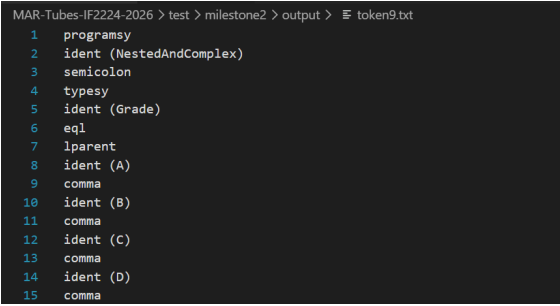
```


Keterangan

Pengujian kedelapan berfokus pada penanganan komentar dan konstanta. Program ini memuat komentar dengan format { ... } dan (* ... *), serta konstanta bernilai positif, negatif, karakter, dan string. Output token menunjukkan bahwa komentar berhasil dikenali oleh lexer sebagai token comment, sedangkan parse tree tidak memasukkan komentar ke dalam struktur sintaks utama karena komentar tidak memengaruhi grammar program. Hasil ini menunjukkan bahwa sistem mampu membedakan elemen yang relevan untuk parsing dari elemen yang hanya bersifat anotatif pada source code.

3.9. Pengujian Kesembilan

Input (input9.txt)	Output (parse_tree9.txt)
<pre>MAR-Tubes-IF2224-2026 > test > milestone2 > input > E input9.txt 1 program NestedAndComplex; 2 type 3 Grade == (A, B, C, D, E); 4 ScoreRange == 0..100; 5 var 6 score, i, j: integer; 7 grade: Grade; 8 matrix: array[1..3] of integer; 9 procedure PrintMatrix(size: integer); 10 var 11 k: integer; 12 begin 13 for k := 1 to size do 14 writeln(matrix[k]) 15 end;</pre> <pre>program NestedAndComplex; type Grade == (A, B, C, D, E); ScoreRange == 0..100; var score, i, j: integer; grade: Grade; matrix: array[1..3] of integer; procedure PrintMatrix(size: integer); var k: integer; begin for k := 1 to size do writeln(matrix[k]) end; function GetGrade(s: integer): integer; begin if s >= 80 then GetGrade := 1 else if s >= 60 then GetGrade := 2 else GetGrade := 3 end; begin for i := 1 to 3 do begin</pre>	<pre>MAR-Tubes-IF2224-2026 > test > milestone2 > output > E parse_tree9.txt 1 <program> 2 <program-header> 3 programsy (program) 4 ident (NestedAndComplex) 5 semicolon (;) 6 <declaration-part> 7 <type-declaration> 8 typesy (type) 9 <type-definition> 10 ident (Grade) 11 eql (==) 12 <type> 13 <enumerated></pre> <pre><program> <program-header> programsy (program) ident (NestedAndComplex) semicolon (;) <declaration-part> <type-declaration> typesy (type) <type-definition> ident (Grade) eql (==) <type> <enumerated> lparent (() <identifier-list> ident (A) comma (,) ident (B) comma (,) ident (C) comma (,) ident (D) comma (,) ident (E) rparent ()) semicolon (;) <type-definition></pre>

<pre> matrix[i] := i * 10; end; PrintMatrix(3); score := 75; case GetGrade(score) of 1: writeln('A'); 2: writeln('B'); 3: writeln('C') end; i := 1; while i <= 3 do begin j := 1; while j <= 3 do begin writeln(i * j); j := j + 1 end; i := i + 1 end end. </pre>	<pre> ident (ScoreRange) eq1 (==) <type> <range> <constant> intcon (0) period (.) period (.) <constant> intcon (100) semicolon (;) <var-declaration> varsy (var) <identifier-list> ident (score) comma (,) ident (i) comma (,) ident (j) colon (:) <type> ident (integer) semicolon (;) <identifier-list> ident (grade) colon (:) <type> ident (Grade) semicolon (;) <identifier-list> ident (matrix) colon (:) <type> <array-type> arraysy (array) lbrack ([]) <range> <constant> intcon (1) period (.) period (.) <constant> intcon (3) rbrack (]) ofsy (of) <type> ident (integer) semicolon (;) <subprogram-declaration-part> <subprogram-declaration> </pre>
Output (token9.txt)	
 <pre> 1 programsy 2 ident (NestedAndComplex) 3 semicolon 4 typesy 5 ident (Grade) 6 eql 7 lparent 8 ident (A) 9 comma 10 ident (B) 11 comma 12 ident (C) 13 comma 14 ident (D) 15 comma </pre> <pre> programsy ident (NestedAndComplex) semicolon typesy ident (Grade) eql lparent ident (A) comma ident (B) comma ident (C) comma ident (D) comma ident (E) rparent semicolon ident (ScoreRange) </pre>	<pre> ident (integer) semicolon (;) <identifier-list> ident (grade) colon (:) <type> ident (Grade) semicolon (;) <identifier-list> ident (matrix) colon (:) <type> <array-type> arraysy (array) lbrack ([]) <range> <constant> intcon (1) period (.) period (.) <constant> intcon (3) rbrack (]) ofsy (of) <type> ident (integer) semicolon (;) <subprogram-declaration-part> <subprogram-declaration> </pre>

eql intcon (0) period period intcon (100) semicolon varsy ident (score) comma ident (i) comma ident (j) colon ident (integer) semicolon ident (grade) colon ident (Grade) semicolon ident (matrix) colon arraysy lbrack intcon (1) period period intcon (3) rbrack ofsy ident (integer) semicolon proceduresy ident (PrintMatrix) lparent ident (size) colon ident (integer) rparent semicolon varsy ident (k) colon ident (integer) semicolon beginsy forsy ident (k) becomes intcon (1) tosy	<procedure-declaration> <procedure-heading> proceduresy (procedure) ident (PrintMatrix) <formal-parameter-list> lparent () <parameter-group> <identifier-list> ident (size) colon (:) ident (integer) rparent () semicolon (;) <block> <declaration-part> <var-declaration> varsy (var) <identifier-list> ident (k) colon (:) <type> ident (integer) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <for-statement> forsy (for) ident (k) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (1) tosy (to) <expression> <simple-expression> <term> <factor> <variable> ident (size) dosy (do) <statement> <procedure/function-call> ident (writeln) lparent () <parameter-list> <expression>
---	--

ident (size)	<simple-expression>
dosy	<term>
ident (writeln)	<factor>
lparent	<variable>
ident (matrix)	ident (matrix)
lbrack	<selector>
ident (k)	lbrack (]
rbrack	<index-list>
rparent	<expression>
endsy	<simple-expression>
semicolon	<term>
functionsy	<factor>
ident (GetGrade)	<variable>
lparent	ident (k)
ident (s)	rbrack (]
colon	rparent ())
ident (integer)	endsy (end)
rparent	semicolon (;)
colon	<subprogram-declaration>
ident (integer)	<function-declaration>
semicolon	<function-heading>
beginsy	functionsy (function)
ifsy	ident (GetGrade)
ident (s)	<formal-parameter-list>
geq	lparent (()
intcon (80)	<parameter-group>
thensy	<identifier-list>
ident (GetGrade)	ident (s)
becomes	colon (:)
intcon (1)	ident (integer)
elsesy	rparent ())
ifsy	colon (:)
ident (s)	ident (integer)
geq	semicolon (;)
intcon (60)	<block>
thensy	<declaration-part>
ident (GetGrade)	<compound-statement>
becomes	beginsy (begin)
intcon (2)	<statement-list>
elsesy	<statement>
ident (GetGrade)	<if-statement>
becomes	ifsy (if)
intcon (3)	<expression>
endsy	<simple-expression>
semicolon	<term>
beginsy	<factor>
forsy	<variable>
ident (i)	ident (s)
becomes	<relational-operator>
intcon (1)	geq (>=)

tosy intcon (3) dosy beginsy ident (matrix) lbrack ident (i) rbrack becomes ident (i) times intcon (10) semicolon endsy semicolon ident (PrintMatrix) lparent intcon (3) rparent semicolon ident (score) becomes intcon (75) semicolon casesy ident (GetGrade) lparent ident (score) rparent ofsy intcon (1) colon ident (writeln) lparent charcon ('A') rparent semicolon intcon (2) colon ident (writeln) lparent charcon ('B') rparent semicolon intcon (3) colon ident (writeln) lparent charcon ('C') rparent	<simple-expression> <term> <factor> intcon (80) thensy (then) <statement> <assignment-statement> <variable> ident (GetGrade) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (1) elsesy (else) <statement> <if-statement> ifsy (if) <expression> <simple-expression> <term> <factor> <variable> ident (s) <relational-operator> geq (>=) <simple-expression> <term> <factor> intcon (60) thensy (then) <statement> <assignment-statement> <variable> ident (GetGrade) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (2) elsesy (else) <statement> <assignment-statement> <variable> ident (GetGrade) becomes (:=) <expression> <simple-expression>
---	---

endsy	<term>
semicolon	<factor>
ident (i)	intcon (3)
becomes	endsy (end)
intcon (1)	semicolon (;)
semicolon	<compound-statement>
whilesy	beginsy (begin)
ident (i)	<statement-list>
leq	<statement>
intcon (3)	<for-statement>
dosy	forsy (for)
beginsy	ident (i)
ident (j)	becomes (:=)
becomes	<expression>
intcon (1)	<simple-expression>
semicolon	<term>
whilesy	<factor>
ident (j)	intcon (1)
leq	tosy (to)
intcon (3)	<expression>
dosy	<simple-expression>
beginsy	<term>
ident (writeln)	<factor>
lparent	intcon (3)
ident (i)	dosy (do)
times	<statement>
ident (j)	<compound-statement>
rparent	beginsy (begin)
semicolon	<statement-list>
ident (j)	<statement>
becomes	<assignment-statement>
ident (j)	<variable>
plus	ident (matrix)
intcon (1)	<selector>
endsy	lbrack ([)
semicolon	<index-list>
ident (i)	<expression>
becomes	<simple-expression>
ident (i)	<term>
plus	<factor>
intcon (1)	<variable>
endsy	ident (i)
endsy	rbrack (])
period	becomes (:=)
	<expression>
	<simple-expression>
	<term>
	<factor>
	<variable>
	ident (i)

	<multiplicative-operator> times (*) <factor> intcon (10) semicolon (;) <statement> <empty> endsy (end) semicolon (;) <statement> <procedure/function-call> ident (PrintMatrix) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> intcon (3) rparent () semicolon (;) <statement> <assignment-statement> <variable> ident (score) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (75) semicolon (;) <statement> <case-statement> casesy (case) <expression> <simple-expression> <term> <factor> <procedure/function-call> ident (GetGrade) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (score) rparent ()
--	--

	ofsy (of) <case-block> <constant> intcon (1) colon (:) <statement> <procedure/function-call> ident (writeln) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> charcon ('A') rparent () semicolon (;) <case-block> <constant> intcon (2) colon (:) <statement> <procedure/function-call> ident (writeln) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> charcon ('B') rparent () semicolon (;) <case-block> <constant> intcon (3) colon (:) <statement> <procedure/function-call> ident (writeln) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> charcon ('C') rparent () endsy (end) semicolon (;)
--	---

	<statement> <assignment-statement> <variable> ident (i) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (1) semicolon (;) <statement> <while-statement> whilesy (while) <expression> <simple-expression> <term> <factor> <variable> ident (i) <relational-operator> leq (<=) <simple-expression> <term> <factor> intcon (3) dosy (do) <statement> <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (j) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (1) semicolon (;) <statement> <while-statement> whilesy (while) <expression> <simple-expression> <term> <factor> <variable>
--	--

	ident (j) <relational-operator> leq (<=) <simple-expression> <term> <factor> intcon (3) dosy (do) <statement> <compound-statement> beginsy (begin) <statement-list> <statement> <procedure/function-call> ident (writeln) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (i) <multiplicative-operator> times (*) <factor> <variable> ident (j) rparent () semicolon (;) <statement> <assignment-statement> <variable> ident (j) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (j) <additive-operator> plus (+) <term> <factor> intcon (1) endsy (end) semicolon (;) <statement>
--	---

	<pre><assignment-statement> <variable> ident (i) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (i) <additive-operator> plus (+) <term> <factor> intcon (1) endsy (end) endsy (end) period (.)</pre>
Keterangan	
Pengujian kesembilan menggabungkan beberapa fitur tingkat lanjut dalam satu program. Kasus ini mencakup deklarasi enumerasi, range, array, prosedur, fungsi, nested if, pemanggilan fungsi di dalam case, serta nested while. Output token menunjukkan bahwa seluruh komponen program kompleks dapat dipecah menjadi deretan token yang lengkap dan berurutan. Parse tree yang dihasilkan memperlihatkan bahwa parser mampu menjaga hierarki struktur program, terutama pada bagian subprogram, statement bersarang, dan ekspresi yang digunakan sebagai parameter.	

3.10. Pengujian Kesepuluh

Input (input10.txt)	Output (parse_tree10.txt)
<pre>MAR-Tubes-IF2224-2026 > test > milestone2 > input > ❯ input10.txt 1 program FullFeatures; 2 const 3 N == 5; 4 TRUEVAL == 1; 5 type 6 Arr == array[1..5] of integer; 7 Direction == (North, South, East, West); 8 Coord == record 9 x: integer; 10 y: integer; 11 end; 12 var 13 nums: Arr; 14 pos: Coord; 15 dir: Direction; 16 i, j, temp, total: integer; 17 procedure BubbleSort(size: integer);</pre> program FullFeatures; const N == 5; TRUEVAL == 1; type Arr == array[1..5] of integer; Direction == (North, South, East, West); Coord == record	<pre>MAR-Tubes-IF2224-2026 > test > milestone2 > output > ❯ parse_tree10.txt 1 <program> 2 <program-header> 3 programsy (program) 4 ident (FullFeatures) 5 semicolon (;) 6 <declaration-part> 7 <const-declaration> 8 constsy (const) 9 <const-definition> 10 ident (N) 11 eql (==) 12 <constant> 13 intcon (5) 14 semicolon (;) 15 <const-definition> 16 ident (TRUEVAL) 17 eql (==)</pre> <program> <program-header> programsy (program) ident (FullFeatures) semicolon (;) <declaration-part>

```

    x: integer;
    y: integer
end;
var
    nums: Arr;
    pos: Coord;
    dir: Direction;
    i, j, temp, total: integer;
procedure BubbleSort(size: integer);
var
    k, l: integer;
begin
    for k := 1 to size - 1 do
        for l := 1 to size - k do
            if nums[l] > nums[l + 1] then begin
                temp := nums[l];
                nums[l] := nums[l + 1];
                nums[l + 1] := temp
            end
        end
    end;
end;
function Sum(size: integer): integer;
var
    k, s: integer;
begin
    s := 0;
    for k := 1 to size do
        s := s + nums[k];
    Sum := s
end;
begin
    nums[1] := 64;
    nums[2] := 34;
    nums[3] := 25;
    nums[4] := 12;
    nums[5] := 22;
    BubbleSort(N);
    total := Sum(N);
    for i := 1 to N do
        writeln(nums[i]);
    writeln(total);
    pos.x := 0;
    pos.y := 0;
    i := 1;
    repeat
        if i mod 2 == 0 then
            pos.x := pos.x + 1
        else
            pos.y := pos.y + 1;
        i := i + 1
    until i > 10;

```

```

<const-declaration>
constsy (const)
<const-definition>
    ident (N)
    eql (==)
    <constant>
        intcon (5)
    semicolon (;)
    <const-definition>
        ident (TRUEVAL)
        eql (==)
        <constant>
            intcon (1)
    semicolon (;)
<type-declaration>
typesy (type)
<type-definition>
    ident (Arr)
    eql (==)
    <type>
        <array-type>
            arraysy (array)
            lbrack ([)
            <range>
                <constant>
                    intcon (1)
                period (.)
                period (.)
                <constant>
                    intcon (5)
            rbrack (])
            ofsy (of)
        <type>
            ident (integer)
    semicolon (;)
    <type-definition>
        ident (Direction)
        eql (==)
        <type>
            <enumerated>
                lparent ((
                <identifier-list>
                    ident (North)
                    comma (,)
                    ident (South)
                    comma (,)
                    ident (East)
                    comma (,)
                    ident (West)
                rparent ())

```

<pre>writeln(pos.x); writeln(pos.y) end.</pre>	
Output (token10.txt)	
<pre>MAR-Tubes-IF2224-2026 > test > milestone2 > output > token10.txt 1 programsy 2 ident (FullFeatures) 3 semicolon 4 constsy 5 ident (N) 6 eql 7 intcon (5) 8 semicolon 9 ident (TRUEVAL) 10 eql 11 intcon (1) 12 semicolon 13 typesy 14 ident (Arr)</pre> <pre> programsy ident (FullFeatures) semicolon constsy ident (N) eql intcon (5) semicolon ident (TRUEVAL) eql intcon (1) semicolon typesy ident (Arr) eql arraysy lbrack intcon (1) period period intcon (5) rbrack ofsy ident (integer) semicolon ident (Direction) eql lparent ident (North) comma ident (South) comma ident (East) comma ident (West) rparent </pre>	<pre> semicolon (;) <type-definition> ident (Coord) eql (==) <type> <record-type> recordsy (record) <field-list> <field-part> <identifier-list> ident (x) colon (:) <type> ident (integer) semicolon (;) <field-part> <identifier-list> ident (y) colon (:) <type> ident (integer) endsy (end) semicolon (;) <var-declaration> varsy (var) <identifier-list> ident (nums) colon (:) <type> ident (Arr) semicolon (;) <identifier-list> ident (pos) colon (:) <type> ident (Coord) semicolon (;) <identifier-list> ident (dir) colon (:) <type> ident (Direction) semicolon (;) <identifier-list> ident (i) comma (,) ident (j) comma (,) ident (temp) comma (,) </pre>

semicolon ident (Coord) eql recordsy ident (x) colon ident (integer) semicolon ident (y) colon ident (integer) endsy semicolon varsy ident (nums) colon ident (Arr) semicolon ident (pos) colon ident (Coord) semicolon ident (dir) colon ident (Direction) semicolon ident (i) comma ident (j) comma ident (temp) comma ident (total) colon ident (integer) semicolon proceduresy ident (BubbleSort) lparent ident (size) colon ident (integer) rparent semicolon varsy ident (k) comma ident (l) colon ident (integer)	ident (total) colon (:) <type> ident (integer) semicolon (;) <subprogram-declaration-part> <subprogram-declaration> <procedure-declaration> <procedure-heading> proceduresy (procedure) ident (BubbleSort) <formal-parameter-list> lparent () <parameter-group> <identifier-list> ident (size) colon (:) ident (integer) rparent () semicolon (;) <block> <declaration-part> <var-declaration> varsy (var) <identifier-list> ident (k) comma (,) ident (l) colon (:) <type> ident (integer) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <for-statement> forsy (for) ident (k) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (1) tosy (to) <expression> <simple-expression> <term> <factor>
---	--

semicolon	<variable>
beginsy	ident (size)
forsy	<additive-operator>
ident (k)	minus (-)
becomes	<term>
intcon (1)	<factor>
tosy	intcon (1)
ident (size)	dosy (do)
minus	<statement>
intcon (1)	<for-statement>
dosy	forsy (for)
forsy	ident (l)
ident (l)	becomes (:=)
becomes	<expression>
intcon (1)	<simple-expression>
tosy	<term>
ident (size)	<factor>
minus	intcon (1)
ident (k)	tosy (to)
dosy	<expression>
ifsy	<simple-expression>
ident (nums)	<term>
lbrack	<factor>
ident (l)	<variable>
rbrack	ident (size)
gtr	<additive-operator>
ident (nums)	minus (-)
lbrack	<term>
ident (l)	<factor>
plus	<variable>
intcon (1)	ident (k)
rbrack	dosy (do)
thensy	<statement>
beginsy	<if-statement>
ident (temp)	ifsy (if)
becomes	<expression>
ident (nums)	<simple-expression>
lbrack	<term>
ident (l)	<factor>
rbrack	<variable>
semicolon	ident (nums)
ident (nums)	<selector>
lbrack	lbrack (l)
ident (l)	<index-list>
rbrack	<expression>
becomes	
ident (nums)	<simple-expression>
lbrack	
ident (l)	<term>
plus	<factor>
	<variable>

intcon (1)	ident (l)
rbrack	rbrack (])
semicolon	<relational-operator>
ident (nums)	gtr (>)
lbrack	<simple-expression>
ident (l)	<term>
plus	<factor>
intcon (1)	<variable>
rbrack	ident (nums)
becomes	<selector>
ident (temp)	lbrack ([)
endsy	<index-list>
endsy	<expression>
semicolon	
functionsy	<simple-expression>
ident (Sum)	<term>
lparent	<factor>
ident (size)	<variable>
colon	ident (l)
ident (integer)	
rparent	<additive-operator>
colon	
ident (integer)	plus (+)
semicolon	<term>
varsy	<factor>
ident (k)	intcon (1)
comma	rbrack (])
ident (s)	thensy (then)
colon	<statement>
ident (integer)	<compound-statement>
semicolon	beginsy (begin)
beginsy	<statement-list>
ident (s)	<statement>
becomes	<assignment-statement>
intcon (0)	<variable>
semicolon	ident (temp)
forsy	becomes (:=)
ident (k)	<expression>
becomes	<simple-expression>
intcon (1)	<term>
tosy	<factor>
ident (size)	<variable>
dosy	ident (nums)
ident (s)	<selector>
becomes	lbrack ([)
ident (s)	<index-list>
plus	<expression>
ident (nums)	
lbrack	<simple-expression>
ident (k)	<term>
	<factor>

rbrack	
semicolon	
ident (Sum)	
becomes	
ident (s)	
endsy	
semicolon	
beginsy	
ident (nums)	
lbrack	
intcon (1)	
rbrack	
becomes	
intcon (64)	
semicolon	
ident (nums)	
lbrack	
intcon (2)	
rbrack	
becomes	
intcon (34)	
semicolon	
ident (nums)	
lbrack	
intcon (3)	
rbrack	
becomes	
intcon (25)	
semicolon	
ident (nums)	
lbrack	
intcon (4)	
rbrack	
becomes	
intcon (12)	
semicolon	
ident (nums)	
lbrack	
intcon (5)	
rbrack	
becomes	
intcon (22)	
semicolon	
ident (BubbleSort)	
lparent	
ident (N)	
rparent	
semicolon	
ident (total)	
becomes	
	<variable>
	ident (l)
	rbrack (l)
	semicolon (;)
	<statement>
	<assignment-statement>
	<variable>
	ident (nums)
	<selector>
	lbrack (l)
	<index-list>
	<expression>
	<simple-expression>
	<term>
	<factor>
	<variable>
	ident (l)
	rbrack (l)
	becomes (:=)
	<expression>
	<simple-expression>
	<term>
	<factor>
	<variable>
	ident (nums)
	<selector>
	lbrack (l)
	<index-list>
	<expression>
	<simple-expression>
	<term>
	<factor>
	<variable>
	ident (l)
	<additive-operator>
	plus (+)
	<term>
	<factor>
	intcon (1)
	rbrack (l)
	semicolon (;)
	<statement>
	<assignment-statement>
	<variable>
	ident (nums)

ident (Sum)	<selector>
lparent	lbrack (l)
ident (N)	<index-list>
rparent	<expression>
semicolon	
forsy	<simple-expression>
ident (i)	<term>
becomes	<factor>
intcon (1)	<variable>
tosy	ident (l)
ident (N)	
dosy	<additive-operator>
ident (writeln)	plus (+)
lparent	<term>
ident (nums)	<factor>
lbrack	intcon (1)
ident (i)	rbrack (r)
rbrack	becomes (:=)
rparent	<expression>
semicolon	<simple-expression>
ident (writeln)	<term>
lparent	<factor>
ident (total)	<variable>
rparent	ident (temp)
semicolon	endsy (end)
ident (pos)	endsy (end)
period	semicolon (;)
ident (x)	<subprogram-declaration>
becomes	<function-declaration>
intcon (0)	<function-heading>
semicolon	functionsy (function)
ident (pos)	ident (Sum)
period	<formal-parameter-list>
ident (y)	lparent (l)
becomes	<parameter-group>
intcon (0)	<identifier-list>
semicolon	ident (size)
ident (i)	colon (:)
becomes	ident (integer)
intcon (1)	rparent (r)
semicolon	colon (:)
repeatsy	ident (integer)
ifsy	semicolon (;)
ident (i)	<block>
imod	<declaration-part>
intcon (2)	<var-declaration>
eq	varsy (var)
intcon (0)	<identifier-list>
thensy	ident (k)
ident (pos)	comma (,)

period ident (x) becomes ident (pos) period ident (x) plus intcon (1) elsesy ident (pos) period ident (y) becomes ident (pos) period ident (y) plus intcon (1) semicolon ident (i) becomes ident (i) plus intcon (1) untilsy ident (i) gtr intcon (10) semicolon ident (writeln) lparent ident (pos) period ident (x) rparent semicolon ident (writeln) lparent ident (pos) period ident (y) rparent endsy period	ident (s) colon (:) <type> ident (integer) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (s) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (0) semicolon (;) <statement> <for-statement> forsy (for) ident (k) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (1) tosy (to) <expression> <simple-expression> <term> <factor> <variable> ident (size) dosy (do) <statement> <assignment-statement> <variable> ident (s) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (s) <additive-operator> plus (+)
--	--

	$\langle \text{term} \rangle$ $\langle \text{factor} \rangle$ $\langle \text{variable} \rangle$ ident (nums) $\langle \text{selector} \rangle$ lbrack ([) $\langle \text{index-list} \rangle$ $\langle \text{expression} \rangle$ $\langle \text{simple-expression} \rangle$ $\langle \text{term} \rangle$ $\langle \text{factor} \rangle$ $\langle \text{variable} \rangle$ ident (k) rbrack (]) semicolon (;) $\langle \text{statement} \rangle$ $\langle \text{assignment-statement} \rangle$ $\langle \text{variable} \rangle$ ident (Sum) becomes (:=) $\langle \text{expression} \rangle$ $\langle \text{simple-expression} \rangle$ $\langle \text{term} \rangle$ $\langle \text{factor} \rangle$ $\langle \text{variable} \rangle$ ident (s) endsy (end) semicolon (;) $\langle \text{compound-statement} \rangle$ beginsy (begin) $\langle \text{statement-list} \rangle$ $\langle \text{statement} \rangle$ $\langle \text{assignment-statement} \rangle$ $\langle \text{variable} \rangle$ ident (nums) $\langle \text{selector} \rangle$ lbrack ([) $\langle \text{index-list} \rangle$ $\langle \text{expression} \rangle$ $\langle \text{simple-expression} \rangle$ $\langle \text{term} \rangle$ $\langle \text{factor} \rangle$ intcon (1) rbrack (]) becomes (:=) $\langle \text{expression} \rangle$ $\langle \text{simple-expression} \rangle$ $\langle \text{term} \rangle$ $\langle \text{factor} \rangle$ intcon (64)
--	---

	semicolon (;) <statement> <assignment-statement> <variable> ident (nums) <selector> lbrack ([) <index-list> <expression> <simple-expression> <term> <factor> intcon (2) rbrack (]) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (34) semicolon (;) <statement> <assignment-statement> <variable> ident (nums) <selector> lbrack ([) <index-list> <expression> <simple-expression> <term> <factor> intcon (3) rbrack (]) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (25) semicolon (;) <statement> <assignment-statement> <variable> ident (nums) <selector> lbrack ([) <index-list> <expression> <simple-expression>
--	--

	$\langle \text{term} \rangle$ $\langle \text{factor} \rangle$ intcon (4) rbrack (]) becomes ($:=$) $\langle \text{expression} \rangle$ $\langle \text{simple-expression} \rangle$ $\langle \text{term} \rangle$ $\langle \text{factor} \rangle$ intcon (12) semicolon (;) $\langle \text{statement} \rangle$ $\langle \text{assignment-statement} \rangle$ $\langle \text{variable} \rangle$ ident (nums) $\langle \text{selector} \rangle$ lbrack ([) $\langle \text{index-list} \rangle$ $\langle \text{expression} \rangle$ $\langle \text{simple-expression} \rangle$ $\langle \text{term} \rangle$ $\langle \text{factor} \rangle$ intcon (5) rbrack (]) becomes ($:=$) $\langle \text{expression} \rangle$ $\langle \text{simple-expression} \rangle$ $\langle \text{term} \rangle$ $\langle \text{factor} \rangle$ intcon (22) semicolon (;) $\langle \text{statement} \rangle$ $\langle \text{procedure/function-call} \rangle$ ident (BubbleSort) lparent (($\langle \text{parameter-list} \rangle$ $\langle \text{expression} \rangle$ $\langle \text{simple-expression} \rangle$ $\langle \text{term} \rangle$ $\langle \text{factor} \rangle$ $\langle \text{variable} \rangle$ ident (N) rparent ()) semicolon (;) $\langle \text{statement} \rangle$ $\langle \text{assignment-statement} \rangle$ $\langle \text{variable} \rangle$ ident (total) becomes ($:=$) $\langle \text{expression} \rangle$
--	---

```

    <simple-expression>
    <term>
    <factor>
    <procedure/function-call>
    ident (Sum)
    lparent ()
    <parameter-list>
    <expression>
    <simple-expression>
    <term>
    <factor>
    <variable>
    ident (N)
    rparent ()
semicolon (;)
<statement>
<for-statement>
forsy (for)
ident (i)
becomes (:=)
<expression>
<simple-expression>
<term>
<factor>
intcon (1)
tosy (to)
<expression>
<simple-expression>
<term>
<factor>
<variable>
ident (N)
dosy (do)
<statement>
<procedure/function-call>
ident (writeln)
lparent ()
<parameter-list>
<expression>
<simple-expression>
<term>
<factor>
<variable>
ident (nums)
<selector>
lbrack ([])
<index-list>
<expression>
<simple-expression>
<term>

```

	<factor> <variable> ident (i) rbrack (]) rparent ()) semicolon (;) <statement> <procedure/function-call> ident (writeln) lparent (<parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (total) rparent ()) semicolon (;) <statement> <assignment-statement> <variable> ident (pos) <selector> period (.) ident (x) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (0) semicolon (;) <statement> <assignment-statement> <variable> ident (pos) <selector> period (.) ident (y) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (0) semicolon (;) <statement> <assignment-statement> <variable>
--	--

	ident (i) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (1) semicolon (;) <statement> <repeat-statement> repeatsy (repeat) <statement-list> <statement> <if-statement> ifsy (if) <expression> <simple-expression> <term> <factor> <variable> ident (i) <multiplicative-operator> imod (mod) <factor> intcon (2) <relational-operator> eql (==) <simple-expression> <term> <factor> intcon (0) thensy (then) <statement> <assignment-statement> <variable> ident (pos) <selector> period (.) ident (x) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (pos) <selector> period (.) ident (x) <additive-operator>
--	---

	plus (+) <term> <factor> intcon (1) elsesy (else) <statement> <assignment-statement> <variable> ident (pos) <selector> period (.) ident (y) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (pos) <selector> period (.) ident (y) <additive-operator> plus (+) <term> <factor> intcon (1) semicolon (;) <statement> <assignment-statement> <variable> ident (i) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (i) <additive-operator> plus (+) <term> <factor> intcon (1) untilsy (until) <expression> <simple-expression> <term> <factor> <variable>
--	--

	<pre> ident (i) <relational-operator> gtr (>) <simple-expression> <term> <factor> intcon (10) semicolon (;) <statement> <procedure/function-call> ident (writeln) lparent (()) <parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (pos) <selector> period (.) ident (x) rparent ()) semicolon (;) <statement> <procedure/function-call> ident (writeln) lparent (()) <parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (pos) <selector> period (.) ident (y) rparent ()) endsy (end) period (.) </pre>
--	--

Keterangan

Pengujian kesepuluh merupakan pengujian komprehensif yang menggabungkan hampir seluruh fitur utama yang telah diuji sebelumnya. Program ini memuat konstanta, type alias, array, enumerasi, record, prosedur, fungsi, nested loop, conditional statement, akses array, akses field record, serta pemanggilan fungsi dan prosedur. Output token menunjukkan bahwa lexer dapat mengenali berbagai jenis token dalam program panjang secara konsisten. Parse tree yang dihasilkan menunjukkan bahwa parser mampu membentuk struktur sintaks lengkap untuk

program berskala lebih besar dan membuktikan integrasi antar aturan grammar berjalan dengan baik.

3.11. Pengujian Kesebelas

Input (input11.txt)	Output (error11.txt)
<pre>repo > test > milestone2 > input > input11.txt 1 program HelloWorld; 2 var 3 a, b: integer; 4 begin 5 a := 5; 6 b := a + 10 7 writeln('Result = ', b); 8 end.</pre>	<pre>test > milestone2 > output > error11.txt 1 [ParseError] Diharapkan 'ENDSY', mendapat 'IDENT' (writeln) 2</pre> [ParseError] Diharapkan 'ENDSY', mendapat 'IDENT' (writeln)
Output (token11.txt)	
<pre>repo > test > milestone2 > output > token11.txt You, 4 minutes ago 1 author (You) 1 programsy 2 ident (HelloWorld) 3 semicolon 4 varsy 5 ident (a) 6 comma 7 ident (b) 8 colon ident (integer) semicolon beginsy ident (a) becomes intcon (5) semicolon ident (b) becomes ident (a) plus</pre>	

intcon (10) ident (writeln) lparent string ('Result = ') comma ident (b) rparent semicolon endsy period	
Keterangan Pengujian kesebelas menunjukkan bagaimana parser melakukan penanganan <i>error</i> pada tingkat sintaks. <i>Test case</i> pada pengujian kesebelas ini dirancang sehingga baris kode Arion dengan operasi aritmatika dan operasi <i>assignment</i> tidak diakhiri oleh titik koma (;). Pesan <i>error</i> tersebut menyatakan bahwa apabila baris keenam yang tidak diakhiri dengan titik koma maka seharusnya token yang valid pada bagian tersebut adalah “end.”, sementara token yang terdeteksi adalah “ident b”.	

3.12. Pengujian Keduabelas

Input (input12.txt)	Output (error12.txt)
<pre> repo > test > milestone2 > input > input12.txt 1 program LoopStatements; 2 var 3 i, sum, factorial: integer; 4 sum := 0; 5 begin 6 i := 1; 7 while i <= 10 do begin 8 sum := sum + i; </pre> program LoopStatements; var i, sum, factorial: integer; sum := 0; begin i := 1; while i <= 10 do begin sum := sum + i; i := i + 1 end; factorial := 1; i := 5; repeat factorial := factorial * i; i := i - 1	<pre> test > milestone2 > output > error12.txt 1 [ParseError] Diharapkan 'COLON', mendapat 'BECOMES' (:=) 2 </pre> [ParseError] Diharapkan 'COLON', mendapat 'BECOMES' (:=)

<pre> until i == 0; for i := 10 downto 1 do writeln(i); writeln(sum); writeln(factorial) end. </pre>	
Output (token12.txt)	
<pre> repo > test > milestone2 > output > token12.txt You, 8 minutes ago 1 author (You) 1 programsy 2 ident (LoopStatements) 3 semicolon 4 varsy 5 ident (i) 6 comma 7 ident (sum) 8 comma 9 ident (factorial) 10 colon 11 ident (integer) 12 semicolon 13 ident (sum) 14 becomes </pre> <pre> programsy ident (LoopStatements) semicolon varsy ident (i) comma ident (sum) comma ident (factorial) colon ident (integer) semicolon ident (sum) becomes intcon (0) semicolon beginsy ident (i) becomes intcon (1) semicolon whilesy ident (i) leq intcon (10) </pre>	

dosy beginsy ident (sum) becomes ident (sum) plus ident (i) semicolon ident (i) becomes ident (i) plus intcon (1) endsy semicolon ident (factorial) becomes intcon (1) semicolon ident (i) becomes intcon (5) semicolon repeatsy ident (factorial) becomes ident (factorial) times ident (i) semicolon ident (i) becomes ident (i) minus intcon (1) untilsy ident (i) eq intcon (0) semicolon forsy ident (i) becomes intcon (10) downtosy intcon (1) dosy ident (writeln) lparent ident (i)	
---	--

rp parent semicolon ident (writeln) lp parent ident (sum) rp parent semicolon ident (writeln) lp parent ident (factorial) rp parent endsy period	
Keterangan	
Pengujian keduabelas menunjukkan bagaimana parser melakukan penanganan <i>error</i> pada tingkat sintaks. <i>Test case</i> pada pengujian kesebelas ini dirancang sehingga baris kode operasi <i>assignment</i> dipindahkan ke bagian deklarasi. Pesan <i>error</i> tersebut menyatakan bahwa pada baris keempat pada <i>source code</i> , yang diharapkan adalah token titik dua, sementara token yang diberikan adalah token “BECOME” atau token “:=”. Kesalahan ini luput pada tingkat <i>lexer</i> , tetapi terdeteksi pada tingkat sintaks atau <i>parser</i> .	

3.13. Pengujian Ketigabelas

Input (input13.txt)	Output (error13.txt)
<pre> repo > test > milestone2 > input > input13.txt 1 program NestedAndComplex; 2 type 3 Grade == (A, B, C, D, E); 4 ScoreRange == 0..100; 5 var 6 score, i, j: integer; 7 grade: Grade; 8 matrix: array[1..3] of integer; 9 procedure PrintMatrix(size: integer); 10 var 11 k: integer; 12 begin </pre> <pre> program NestedAndComplex; type Grade == (A, B, C, D, E); ScoreRange == 0..100; var score, i, j: integer; grade: Grade; matrix: array[1..3] of integer; procedure PrintMatrix(size: integer); var k: integer; </pre>	<pre> test>milestone2>output>P error13.txt 1 [ParseError] Body for harus berupa compound-statement: gunakan 'begin ... end' setelah 'do'. Mendapat 'IDENT' 2 </pre> [ParseError] Body for harus berupa compound-statement: gunakan 'begin ... end' setelah 'do'. Mendapat 'IDENT'


```

begin
  for k := 1 to size do
    writeln(matrix[k])
  end;
function GetGrade(s: integer): integer;
begin
  if s >= 80 then
    GetGrade := 1
  else if s >= 60 then
    GetGrade := 2
  else
    GetGrade := 3
  end;
begin
  for i := 1 to 3 do begin
    matrix[i] := i * 10;
  PrintMatrix(3);
  score := 75;
  case GetGrade(score) of
    1: writeln('A');
    2: writeln('B');
    3: writeln('C')
  end;
  i := 1;
  while i <= 3 do begin
    j := 1;
    while j <= 3 do begin
      writeln(i * j);
      j := j + 1
    end
    i := i + 1
  end
end.

```

Output (token13.txt)

repo > test > milestone2 > output > token13.txt

You, 8 minutes ago | 1 author (You)

```
1  programsy
2  ident (NestedAndComplex)
3  semicolon
4  typesy
5  ident (Grade)
6  eql
7  lparent
8  ident (A)
9  comma
10 ident (B)
11 comma
12 ident (C)
13 comma
14 ident (D)
15 comma
16 ident (E)
```

programsy
ident (NestedAndComplex)
semicolon
typesy
ident (Grade)
eql
lparent
ident (A)
comma
ident (B)
comma
ident (C)
comma
ident (D)
comma
ident (E)
rparent
semicolon
ident (ScoreRange)
eql
intcon (0)
period
period
intcon (100)
semicolon
varsy
ident (score)
comma
ident (i)
comma
ident (j)
colon

ident (integer) semicolon ident (grade) colon ident (Grade) semicolon ident (matrix) colon arraysy lbrack intcon (1) period period intcon (3) rbrack ofsy ident (integer) semicolon proceduresy ident (PrintMatrix) lparent ident (size) colon ident (integer) rparent semicolon varsy ident (k) colon ident (integer) semicolon beginsy forsy ident (k) becomes intcon (1) tosy ident (size) dosy ident (writeln) lparent ident (matrix) lbrack ident (k) rbrack rparent endsy semicolon functionsy ident (GetGrade)	
--	--

lparent ident (s) colon ident (integer) rparent colon ident (integer) semicolon beginsy ifsy ident (s) geq intcon (80) thensy ident (GetGrade) becomes intcon (1) elsesy ifsy ident (s) geq intcon (60) thensy ident (GetGrade) becomes intcon (2) elsesy ident (GetGrade) becomes intcon (3) endsy semicolon beginsy forsy ident (i) becomes intcon (1) tosy intcon (3) dosy beginsy ident (matrix) lbrack ident (i) rbrack becomes ident (i) times intcon (10) semicolon	
---	--

ident (PrintMatrix) lparent intcon (3) rparent semicolon ident (score) becomes intcon (75) semicolon casesy ident (GetGrade) lparent ident (score) rparent ofsy intcon (1) colon ident (writeln) lparent charcon ('A') rparent semicolon intcon (2) colon ident (writeln) lparent charcon ('B') rparent semicolon intcon (3) colon ident (writeln) lparent charcon ('C') rparent endsy semicolon ident (i) becomes intcon (1) semicolon whilesy ident (i) leq intcon (3) dosy beginsy ident (j) becomes intcon (1)	
---	--

semicolon whilesy ident (j) leq intcon (3) dosy beginsy ident (writeln) lparent ident (i) times ident (j) rparent semicolon ident (j) becomes ident (j) plus intcon (1) ident (i) becomes ident (i) plus intcon (1) endsy endsy period	
Keterangan	
Pengujian ketigabelas menunjukkan bagaimana parser melakukan penanganan <i>error</i> pada tingkat sintaks. <i>Test case</i> pada pengujian ketigabelas ini dirancang sehingga salah satu token “ENDSY” atau “end.” pada baris ke-41 dihilangkan dari blok “while”. Pesan error tersebut menyatakan bahwa pada baris ke-41 tersebut token yang diharapkan sesuai aturan produksi adalah “ENDSY” atau “end.”, tetapi token yang diberikan oleh <i>source code</i> melalui <i>lexer</i> adalah “IDENT i” sebagaimana baris ke-41 adalah potongan kode berikut “i := i + 1”.	

3.14. Pengujian Keempatbelas

Input (input14.txt)	Output (error14.txt)
<pre>test > milestone2 > input >  input14.txt 1 program ExtraToken; 2 begin 3 end. 4 junk 5 </pre>	<pre>test > milestone2 > output >  error14.txt 1 [ParseError] Token tambahan setelah akhir program: 'IDENT' (junk) 2</pre> [ParseError] Token tambahan setelah akhir program: 'IDENT' (junk)

Output (token14.txt) <pre> ≡ token14.txt X test > milestone2 > output > ≡ token14.txt 1 programsy 2 ident (ExtraToken) 3 semicolon 4 beginsy 5 endsy 6 period 7 ident (junk) 8 </pre>	
Keterangan Pengujian keempatbelas digunakan untuk memastikan parser mampu mendeteksi token tambahan setelah akhir program. Input ini memiliki struktur program yang sebenarnya sudah selesai pada token period setelah end., tetapi setelah itu masih terdapat token tambahan berupa identifier junk. Kasus ini penting karena parser tidak boleh hanya menerima prefix program yang valid. Parser harus memastikan bahwa seluruh token pada input telah dikonsumsi sesuai grammar. Apabila masih terdapat token setelah struktur <program> selesai dibaca, maka input harus dianggap tidak valid. Hasil pengujian menunjukkan bahwa lexer tetap dapat menghasilkan token untuk seluruh input, termasuk token tambahan setelah period. Namun, parser menolak input tersebut karena terdapat token sisa setelah akhir program. Dengan demikian, pengujian ini membuktikan bahwa parser mampu mendeteksi extra token setelah program selesai.	

3.15. Pengujian Kelimabelas

Input (input15.txt) <pre> test > milestone2 > input > ≡ input15.txt 1 program InvalidDeclarationOrder; 2 var 3 a: integer; 4 const 5 C == 1; 6 begin 7 end. 8 </pre>	Output (error15.txt) <pre> test > milestone2 > output > ≡ error15.txt 1 [ParseError] Urutan deklarasi tidak valid. Deklarasi harus berurutan: const, type, var, procedure/function. Mendapat 'CONSTSY' 2 </pre> [ParseError] Urutan deklarasi tidak valid. Deklarasi harus berurutan: const, type, var, procedure/function. Mendapat 'CONSTSY'
Output (token1.txt)	

<pre> test > milestone2 > output > token15.txt 1 programsy 2 ident (InvalidDeclarationOrder) 3 semicolon 4 varsy 5 ident (a) 6 colon 7 ident (integer) 8 semicolon 9 constsy 10 ident (C) 11 eql 12 intcon (1) 13 semicolon 14 beginsy 15 endsy 16 period 17 </pre>	
Keterangan Pengujian kelimabelas digunakan untuk menguji validasi urutan deklarasi pada grammar Arion. Input ini mendeklarasikan variabel terlebih dahulu menggunakan bagian var, kemudian setelah itu mencoba mendeklarasikan konstanta menggunakan bagian const. Berdasarkan grammar yang digunakan, bagian deklarasi memiliki urutan tertentu, yaitu deklarasi konstanta, deklarasi tipe, deklarasi variabel, dan subprogram. Oleh karena itu, deklarasi const tidak boleh muncul setelah bagian var selesai diproses. Hasil pengujian menunjukkan bahwa lexer berhasil mengenali token-token pada input, tetapi parser menolak struktur program karena urutan deklarasi tidak sesuai grammar. Dengan demikian, pengujian ini membuktikan bahwa parser tidak hanya mengenali bentuk deklarasi secara individual, tetapi juga memvalidasi posisi dan urutannya dalam blok deklarasi program.	

3.16. Pengujian Keenambelas

Input (input16.txt)	Output (parse_tree16.txt)
---------------------	---------------------------


```
test > milestone2 > input > ≡ input16.txt
1  program ArrayIdentIndex;
2  var
3  |  a: array[char] of integer;
4  begin
5  end.
6
```

Output (token16.txt)

```
test > milestone2 > output > ≡ token16.txt
1  programsy
2  ident (ArrayIdentIndex)
3  semicolon
4  varsy
5  ident (a)
6  colon
7  arraysy
8  lbrack
9  ident (char)
10 rbrack
11 ofsy
12 ident (integer)
13 semicolon
14 beginsy
15 endsy
16 period
17
```

programsy
 ident (HelloWorld)
 semicolon
 varsy
 ident (a)
 comma
 ident (b)
 colon
 ident (integer)
 semicolon
 beginsy
 ident (a)
 becomes
 intcon (5)
 semicolon
 ident (b)
 becomes
 ident (a)
 plus
 intcon (10)
 semicolon

```
test > milestone2 > output > ≡ parse_tree16.txt
1  <program>
2  <program-header>
3  programsy (program)
4  ident (ArrayIdentIndex)
5  semicolon (;)
6  <declaration-part>
7  <var-declaration>
8  varsy (var)
9  <identifier-list>
10 ident (a)
11 colon (:)
12 <type>
13 <array-type>
14 arraysy (array)
15 lbrack ([)
16 ident (char)
17 rbrack (])
18 ofsy (of)
19 <type>
20 ident (integer)
21 semicolon (;)
22 <compound-statement>
23 beginsy (begin)
24 <statement-list>
25 <statement>
26 <empty>
27 endsy (end)
28 period (.)
```

ident (writeln) lparent string ('Result = ') comma ident (b) rparent semicolon endsy period	
Keterangan	
Pengujian keenambelas digunakan untuk memeriksa kemampuan parser dalam mengenali deklarasi array dengan tipe indeks berupa identifier. Pada input ini, variabel a dideklarasikan sebagai array[char] of integer. Kasus ini menguji apakah parser mampu membaca bentuk array-type yang indeksinya bukan hanya range eksplisit seperti 1..10, tetapi juga tipe bernama atau tipe bawaan seperti char. Dalam grammar, bagian indeks array harus dapat dikenali sebagai bagian dari struktur tipe array. Hasil pengujian menunjukkan bahwa parser berhasil membentuk parse tree untuk deklarasi array tersebut. Parse tree yang dihasilkan memuat node untuk deklarasi variabel, tipe array, indeks array, dan tipe elemen. Dengan demikian, parser mampu mengenali deklarasi array dengan tipe indeks berbentuk identifier.	

3.17. Pengujian Ketujuhbelas

Input (input17.txt)	Output (parse_tree17.txt)
----------------------------	----------------------------------

```
test > milestone2 > input > ≡ input17.txt
```

```
1 program ArrayRangeIndex;  
2 var  
3   a: array[1..10] of integer;  
4 begin  
5 end.
```

Output (token17.txt)

```
test > milestone2 > output > ≡ token17.txt
```

```
1 programsy  
2 ident (ArrayRangeIndex)  
3 semicolon  
4 varsy  
5 ident (a)  
6 colon  
7 arraysy  
8 lbrack  
9 intcon (1)  
10 period  
11 period  
12 intcon (10)  
13 rbrack  
14 ofsy  
15 ident (integer)  
16 semicolon  
17 beginsy  
18 endsy  
19 period  
20
```

```
test > milestone2 > output > ≡ parse_tree17.txt
```

```
1 <program>  
2 <program-header>  
3 programsy (program)  
4 ident (ArrayRangeIndex)  
5 semicolon (;)  
6 <declaration-part>  
7 <var-declaration>  
8 varsy (var)  
9 <identifier-list>  
10 ident (a)  
11 colon (:)  
12 <type>  
13 <array-type>  
14 arraysy (array)  
15 lbrack ([  
16 <range>  
17 <constant>  
18 intcon (1)  
19 period (.)  
20 period (.)  
21 <constant>  
22 intcon (10)  
23 rbrack (])  
24 ofsy (of)  
25 <type>  
26 ident (integer)  
27 semicolon (;)  
28 <compound-statement>  
29 beginsy (begin)  
30 <statement-list>  
31 <statement>  
32 <empty>  
33 endsy (end)  
34 period (.)
```

Keterangan

Pengujian ketujuhbelas digunakan untuk memeriksa deklarasi array dengan indeks berupa range. Pada input ini, variabel a dideklarasikan sebagai array[1..10] of integer.

Kasus ini penting karena range menggunakan dua token period secara berurutan untuk membentuk batas bawah dan batas atas array. Parser harus mampu membedakan penggunaan titik pada range 1..10 dari token period yang digunakan untuk mengakhiri program.

Hasil pengujian menunjukkan bahwa lexer menghasilkan token intcon, period, period, dan intcon untuk range tersebut, lalu parser berhasil menyusunnya menjadi node <range> dalam parse tree. Dengan demikian, parser mampu mengenali array dengan indeks range secara benar.

3.18. Pengujian Kedelapanbelas

Input (input18.txt)	Output (parse_tree18.txt)
<pre> test > milestone2 > input > ≡ input18.txt 1 program ComponentChain; 2 type 3 Inner == record 4 value: integer; 5 end; 6 Item == record 7 inner: Inner; 8 end; 9 var 10 items: array[1..10] of Item; 11 x: integer; 12 begin 13 x := items[1].inner.value; 14 end. 15 </pre>	<pre> test > milestone2 > output > ≡ parse_tree18.txt 1 <program> 2 <program-header> 3 programsy (program) 4 ident (ComponentChain) 5 semicolon (;) 6 <declaration-part> 7 <type-declaration> 8 typesy (type) 9 <type-definition> 10 ident (Inner) 11 eql (==) 12 <type> 13 <record-type> 14 recordsy (record) </pre>
Output (token18.txt)	
<pre> test > milestone2 > output > ≡ token18.txt 1 programsy 2 ident (ComponentChain) 3 semicolon 4 typesy 5 ident (Inner) 6 eql 7 recordsy 8 ident (value) 9 colon 10 ident (integer) 11 semicolon 12 endsy 13 semicolon 14 ident (Item) 15 eql 16 recordsy 17 ident (inner) 18 colon 19 ident (Inner) 20 semicolon </pre> programsy ident (ComponentChain) semicolon typesy ident (Inner) eql recordsy ident (value) colon ident (integer) semicolon endsy semicolon ident (Item) eql recordsy	<pre> <program> <program-header> programsy (program) ident (ComponentChain) semicolon (;) <declaration-part> <type-declaration> typesy (type) <type-definition> ident (Inner) eql (==) <type> <record-type> recordsy (record) <field-list> <field-part> <identifier-list> ident (value) colon (:) <type> ident (integer) semicolon (;) endsy (end) semicolon (;) <type-definition> ident (Item) eql (==) <type> <record-type> recordsy (record) <field-list> <field-part> <identifier-list> ident (inner) colon (:) <type> </pre>

ident (inner) colon ident (Inner) semicolon endsy semicolon varsy ident (items) colon arraysy lbrack intcon (1) period period intcon (10) rbrack ofsy ident (Item) semicolon ident (x) colon ident (integer) semicolon beginsy ident (x) becomes ident (items) lbrack intcon (1) rbrack period ident (inner) period ident (value) semicolon endsy period	ident (Inner) semicolon (;) endsy (end) semicolon (;) <var-declaration> varsy (var) <identifier-list> ident (items) colon (:) <type> <array-type> arraysy (array) lbrack (lbrack) <range> <constant> intcon (1) period (.) period (.) <constant> intcon (10) rbrack (rbrack) ofsy (of) <type> ident (Item) semicolon (;) <identifier-list> ident (x) colon (:) <type> ident (integer) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (x) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (items) <component-variable> lbrack (lbrack) <index-list> <expression> <simple-expression>
--	--

	<pre> <term> <factor> intcon (1) rbrack (]) <component-variable> period (.) ident (inner) <component-variable> period (.) ident (value) semicolon (;) <statement> <empty> endsy (end) period (.) </pre>
Keterangan	
Pengujian kedelapanbelas digunakan untuk menguji struktur tipe kompleks berupa record bertingkat dan array of record. Input ini mendefinisikan tipe Inner sebagai record yang memiliki field value, lalu tipe Item sebagai record yang memiliki field inner bertipe Inner. Setelah itu, variabel items dideklarasikan sebagai array[1..10] of Item. Bagian statement pada program menggunakan akses komponen berantai, yaitu items[1].inner.value. Kasus ini penting karena parser harus mampu mengenali kombinasi selector array dan selector field record secara berurutan. Hasil parse tree menunjukkan bahwa parser berhasil membentuk struktur <selector> untuk akses indeks [1], kemudian selector field .inner, dan selector field .value. Dengan demikian, pengujian ini membuktikan bahwa parser mampu menangani component chain pada array dan record.	

3.19. Pengujian Kesembilanbelas

Input (input19.txt)	Output (parse_tree19.txt)
<pre> test > milestone2 > input > ≡ input19.txt 1 program IndexExpressionExtension; 2 var 3 a: array[1..10] of integer; 4 i, x: integer; 5 begin 6 i := 1; 7 x := a[i + 1]; 8 end. 9 </pre>	<pre> test > milestone2 > output > ≡ parse_tree19.txt 1 <program> 2 <program-header> 3 programsy (program) 4 ident (IndexExpressionExtension) 5 semicolon (;) 6 <declaration-part> 7 <var-declaration> 8 varsy (var) 9 <identifier-list> 10 ident (a) 11 colon (:) 12 <type> 13 <array-type> 14 arraysy (array) 15 lbrack ([) 16 <range> </pre>
Output (token19.txt)	<pre> <program> <program-header> </pre>

test > milestone2 > output > token19.txt

```
1  programsy
2  ident (IndexExpressionExtension)
3  semicolon
4  varsy
5  ident (a)
6  colon
7  arraysy
8  lbrack
9  intcon (1)
10 period
11 period
12 intcon (10)
```

programsy
ident (IndexExpressionExtension)
semicolon
varsy
ident (a)
colon
arraysy
lbrack
intcon (1)
period
period
intcon (10)
rbrack
ofsy
ident (integer)
semicolon
ident (i)
comma
ident (x)
colon
ident (integer)
semicolon
beginsy
ident (i)
becomes
intcon (1)
semicolon
ident (x)
becomes
ident (a)
lbrack
ident (i)
plus
intcon (1)
rbrack
semicolon
endsy

programsy (program)
ident (IndexExpressionExtension)
semicolon (;)
<declaration-part>
<var-declaration>
varsy (var)
<identifier-list>
ident (a)
colon (:)
<type>
<array-type>
arraysy (array)
lbrack ([)
<range>
<constant>
intcon (1)
period (.)
period (.)
<constant>
intcon (10)
rbrack (])
ofsy (of)
<type>
ident (integer)
semicolon (;)
<identifier-list>
ident (i)
comma (,)
ident (x)
colon (:)
<type>
ident (integer)
semicolon (;)
<compound-statement>
beginsy (begin)
<statement-list>
<statement>
<assignment-statement>
<variable>
ident (i)
becomes (:=)
<expression>
<simple-expression>
<term>
<factor>
intcon (1)
semicolon (;)
<statement>
<assignment-statement>
<variable>

period	<pre> ident (x) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (a) <component-variable> lbrack ([<index-list> <expression> <simple-expression> <term> <factor> <variable> ident (i) <additive-operator> plus (+) <term> <factor> intcon (1) rbrack (]) semicolon (;) <statement> <empty> endsy (end) period (.) </pre>
Keterangan Pengujian kesembilanbelas digunakan untuk menguji akses array dengan indeks berupa ekspresi. Input ini mendeklarasikan array a, variabel i, dan variabel x, lalu melakukan assignment $x := a[i + 1]$; Kasus ini penting karena indeks array tidak hanya berupa konstanta literal, tetapi dapat berupa ekspresi yang terdiri dari identifier, operator, dan konstanta. Parser harus mampu membentuk node <code><index-list></code> yang berisi <code><expression></code>, bukan hanya menerima indeks sederhana. Hasil pengujian menunjukkan bahwa parser berhasil mengenali $i + 1$ sebagai ekspresi di dalam selector array. Parse tree yang terbentuk menunjukkan struktur <code><index-list></code>, <code><expression></code>, <code><simple-expression></code>, dan operator penjumlahan. Dengan demikian, parser mampu menangani indeks array berbasis ekspresi.	

3.20. Pengujian Keduapuluh

Input (input20.txt)	Output (parse_tree20.txt)
---------------------	---------------------------


```
test > milestone2 > input > ≡ input20.txt
1  program ValidDeclarationOrder;
2  const
3    A == 1;
4  type
5    T == integer;
6  var
7    x: T;
8  begin
9    x := A;
10 end.
```

Output (token20.txt)

```
test > milestone2 > output > ≡ token20.txt
1  programsy
2  ident (ValidDeclarationOrder)
3  semicolon
4  constsy
5  ident (A)
6  eql
7  intcon (1)
8  semicolon
9  typesy
10 ident (T)
11 eql
12 ident (integer)
13 semicolon
14 varsy
15 ident (x)
16 colon
17 ident (T)
18 semicolon
19 beginsy
20 ident (x)
21 becomes
22 ident (A)
23 semicolon
24 endsy
25 period
```

```
test > milestone2 > output > ≡ parse_tree20.txt
1  <program>
2    <program-header>
3      programsy (program)
4        ident (ValidDeclarationOrder)
5        semicolon (;)
6    <declaration-part>
7      <const-declaration>
8        constsy (const)
9        <const-definition>
10         ident (A)
11         eql (==)
12         <constant>
13         intcon (1)
14         semicolon (;)
15      <type-declaration>
16        typesy (type)
17        <type-definition>
18         ident (T)
19         eql (==)
20         <type>
21         ident (integer)
22         semicolon (;)
23      <var-declaration>
24        varsy (var)
25        <identifier-list>
26         ident (x)
27         colon (:)
28         <type>
29         ident (T)
30         semicolon (;)
31      <compound-statement>
32        beginsy (begin)
33        <statement-list>
```

```
test > milestone2 > output > ≡ parse_tree20.txt
1  <program>
31 <compound-statement>
32   beginsy (begin)
33   <statement-list>
34     <statement>
35       <assignment-statement>
36         <variable>
37         ident (x)
38         becomes (:=)
39         <expression>
40         <simple-expression>
41         <term>
42         <factor>
43         <variable>
44         ident (A)
45         semicolon (;)
46       <statement>
47       <empty>
48     endsy (end)
49   period (.)
```

Keterangan

Pengujian kedua puluh digunakan untuk memastikan parser menerima urutan deklarasi yang valid. Input ini memuat deklarasi konstanta A, deklarasi tipe T, deklarasi variabel x, lalu compound statement yang mengisi x dengan nilai A.

Kasus ini menjadi pembanding dari testcase invalid yang menempatkan deklarasi dalam urutan salah. Pada input ini, deklarasi ditulis sesuai urutan grammar, yaitu const, kemudian type, kemudian var, dan setelah itu bagian begin ... end.

Hasil pengujian menunjukkan bahwa parser berhasil membentuk parse tree secara lengkap. Node deklarasi konstanta, deklarasi tipe, deklarasi variabel, dan assignment statement terbentuk sesuai grammar. Dengan demikian, pengujian ini membuktikan bahwa parser menerima urutan deklarasi yang valid dan dapat memproses penggunaan konstanta serta tipe buatan user dalam program.

3.21. Pengujian Keduapuluhsatu

Input (input21.txt)	Output (parse_tree21.txt)
<pre> test > milestone2 > input > ≡ input21.txt 1 program Hello; 2 var 3 a, b: integer; 4 begin 5 a := 5; 6 b := a + 10; 7 writeln('Result = ', b); 8 end. 9 </pre>	<pre> test > milestone2 > output > ≡ parse_tree21.txt 1 <program> 2 <program-header> 3 programsy (program) 4 ident (Hello) 5 semicolon (;) 6 <declaration-part> 7 <var-declaration> 8 varsy (var) 9 <identifier-list> 10 ident (a) 11 comma (,) 12 ident (b) 13 colon (:) 14 <type> </pre>
Output (token21.txt)	<pre> <program> <program-header> programsy (program) ident (Hello) semicolon (;) <declaration-part> <var-declaration> varsy (var) <identifier-list> ident (a) comma (,) ident (b) colon (:) <type> ident (integer) semicolon (;) <compound-statement> </pre>

test > milestone2 > output > token21.txt

```
1  programsy
2  ident (Hello)
3  semicolon
4  varsy
5  ident (a)
6  comma
7  ident (b)
8  colon
9  ident (integer)
10 semicolon
11 beginsy
12 ident (a)
13 becomes
14 intcon (5)
15 semicolon
16 ident (b)
17 becomes
18 ident (a)
19 plus
20 intcon (10)
21 semicolon
22 ident (writeln)
23 lparent
24 string ('Result = ')
25 comma
26 ident (b)
27 rparent
28 semicolon
29 endsy
30 period
31
```

```
beginsy (begin)
<statement-list>
  <statement>
    <assignment-statement>
      <variable>
        ident (a)
      becomes (:=)
      <expression>
        <simple-expression>
          <term>
            <factor>
              intcon (5)
          semicolon (;)
        <statement>
          <assignment-statement>
            <variable>
              ident (b)
            becomes (:=)
            <expression>
              <simple-expression>
                <term>
                  <factor>
                    <variable>
                      ident (a)
                    <additive-operator>
                      plus (+)
                <term>
                  <factor>
                    intcon (10)
              semicolon (;)
            <statement>
              <procedure/function-call>
                ident (writeln)
                lparent (())
                <parameter-list>
                  <expression>
                    <simple-expression>
                      <term>
                        <factor>
                          string ('Result = ')
                    comma (,)
                  <expression>
                    <simple-expression>
                      <term>
                        <factor>
                          <variable>
                            ident (b)
                      rparent ())
                semicolon (;)
            <statement>
```

	<empty> endsy (end) period (.)
Keterangan	
Pengujian ini digunakan sebagai regression test dasar parser. Program memuat struktur utama Arion berupa program header, deklarasi variabel, compound statement, assignment, ekspresi aritmatika, dan pemanggilan writeln. Tujuan pengujian ini adalah memastikan parser tetap dapat membentuk parse tree untuk program sederhana setelah penambahan mode --parse-only dan --from-tokens. Hasil token menunjukkan bahwa lexer berhasil menghasilkan token yang sesuai. Parse tree yang terbentuk menunjukkan bahwa parser dapat mengenali node <program>, <program-header>, <declaration-part>, <compound-statement>, <statement>, dan <expression> dengan benar.	

3.22. Pengujian Keduapuluhdua

Input (input22.txt)	Output (parse_tree22.txt)
<pre> test > milestone2 > input > ≡ input22.txt 1 program LoopValid; 2 var 3 i, sum: integer; 4 begin 5 i := 0; 6 sum := 0; 7 while i < 3 do begin 8 sum := sum + i; 9 i := i + 1; 10 end; 11 for i := 1 to 3 do begin 12 sum := sum + i; 13 end; 14 writeln(sum); 15 end. </pre>	<pre> test > milestone2 > output > ≡ parse_tree22.txt 1 <program> 2 <program-header> 3 programsy (program) 4 ident (LoopValid) 5 semicolon (;) 6 <declaration-part> 7 <var-declaration> 8 varsy (var) 9 <identifier-list> 10 ident (i) 11 comma (,) 12 ident (sum) 13 colon (:) 14 <type> </pre>
Output (token22.txt)	<pre> <program> <program-header> programsy (program) ident (LoopValid) semicolon (;) <declaration-part> <var-declaration> varsy (var) <identifier-list> ident (i) comma (,) ident (sum) colon (:) <type> ident (integer) semicolon (;) </pre>

```
test > milestone2 > output > token22.txt
```

```
1  programsy
2  ident (LoopValid)
3  semicolon
4  varsy
5  ident (i)
6  comma
7  ident (sum)
8  colon
9  ident (integer)
10 semicolon
11 beginsy
12 ident (i)
13 becomes
14 intcon (0)
```

programsy
ident (LoopValid)
semicolon
varsy
ident (i)
comma
ident (sum)
colon
ident (integer)
semicolon
beginsy
ident (i)
becomes
intcon (0)
semicolon
ident (sum)
becomes
intcon (0)
semicolon
whilesy
ident (i)
lss
intcon (3)
dosy
beginsy
ident (sum)
becomes
ident (sum)
plus
ident (i)
semicolon
ident (i)
becomes
ident (i)

<compound-statement>
beginsy (begin)
<statement-list>
<statement>
<assignment-statement>
<variable>
ident (i)
becomes (:=)
<expression>
<simple-expression>
<term>
<factor>
intcon (0)
semicolon (;)
<statement>
<assignment-statement>
<variable>
ident (sum)
becomes (:=)
<expression>
<simple-expression>
<term>
<factor>
intcon (0)
semicolon (;)
<statement>
<while-statement>
whilesy (while)
<expression>
<simple-expression>
<term>
<factor>
<variable>
ident (i)
<relational-operator>
lss (<)
<simple-expression>
<term>
<factor>
intcon (3)
dosy (do)
<compound-statement>
beginsy (begin)
<statement-list>
<statement>
<assignment-statement>
<variable>
ident (sum)
becomes (:=)
<expression>

plus intcon (1) semicolon endsy semicolon forsy ident (i) becomes intcon (1) tosy intcon (3) dosy beginsy ident (sum) becomes ident (sum) plus ident (i) semicolon endsy semicolon ident (writeln) lparent ident (sum) rparent semicolon endsy period	<simple-expression> <term> <factor> <variable> ident (sum) <additive-operator> plus (+) <term> <factor> <variable> ident (i) semicolon (;) <statement> <assignment-statement> <variable> ident (i) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (i) <additive-operator> plus (+) <term> <factor> intcon (1) semicolon (;) <statement> <empty> endsy (end) semicolon (;) <statement> <for-statement> forsy (for) ident (i) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (1) tosy (to) <expression> <simple-expression> <term> <factor> intcon (3) dosy (do)
--	---

	<pre> <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (sum) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (sum) <additive-operator> plus (+) <term> <factor> <variable> ident (i) semicolon (;) <statement> <empty> endsy (end) semicolon (;) <statement> <procedure/function-call> ident (writeln) lparent (()) <parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (sum) rparent ()) semicolon (;) <statement> <empty> endsy (end) period (.) </pre>
Keterangan	
Pengujian ini berfokus pada struktur perulangan while dan for yang menggunakan compound-statement setelah keyword do. Kasus ini penting karena implementasi terbaru mewajibkan body while dan for berbentuk begin ... end.	

Hasil parse tree menunjukkan bahwa parser mampu mengenali while-statement dan for-statement dengan body berupa <compound-statement>. Dengan demikian, pengujian ini membuktikan bahwa grammar terbaru untuk perulangan sudah diterapkan dengan benar.

3.23. Pengujian Keduapuluhtiga

Input (input23.txt)	Output (parse_tree23.txt)
<pre>test > milestone2 > input > ≡ input23.txt 1 program ArrayRecord; 2 type 3 Person == record 4 name: string; 5 age: integer 6 end; 7 var 8 arr: array[1..5] of integer; 9 p: Person; 10 begin 11 arr[1] := 10; 12 arr[2] := arr[1] + 5; 13 p.age := arr[2]; 14 end. 15</pre>	<pre>test > milestone2 > output > ≡ parse_tree23.txt 1 <program> 2 <program-header> 3 programsy (program) 4 ident (ArrayRecord) 5 semicolon (;) 6 <declaration-part> 7 <type-declaration> 8 typesy (type) 9 <type-definition> 10 ident (Person) 11 eql (==) 12 <type> 13 <record-type> 14 recordsy (record)</pre>
Output (token23.txt)	
<pre>test > milestone2 > output > ≡ token23.txt 1 programsy 2 ident (ArrayRecord) 3 semicolon 4 typesy 5 ident (Person) 6 eql 7 recordsy 8 ident (name) 9 colon 10 ident (string) 11 semicolon 12 ident (age) 13 colon 14 ident (integer)</pre> programsy ident (ArrayRecord) semicolon typesy ident (Person) eql	<pre><program> <program-header> programsy (program) ident (ArrayRecord) semicolon (;) <declaration-part> <type-declaration> typesy (type) <type-definition> ident (Person) eql (==) <type> <record-type> recordsy (record) <field-list> <field-part> <identifier-list> ident (name) colon (:) <type> ident (string) semicolon (;) <field-part> <identifier-list> ident (age) colon (:) <type> ident (integer) endsy (end)</pre>

recordsy ident (name) colon ident (string) semicolon ident (age) colon ident (integer) endsy semicolon varsy ident (arr) colon arraysy lbrack intcon (1) period period intcon (5) rbrack ofsy ident (integer) semicolon ident (p) colon ident (Person) semicolon beginsy ident (arr) lbrack intcon (1) rbrack becomes intcon (10) semicolon ident (arr) lbrack intcon (2) rbrack becomes ident (arr) lbrack intcon (1) rbrack plus intcon (5) semicolon ident (p) period ident (age)	semicolon (;) <var-declaration> varsy (var) <identifier-list> ident (arr) colon (:) <type> <array-type> arraysy (array) lbrack (] <range> <constant> intcon (1) period (.) period (.) <constant> intcon (5) rbrack (] ofsy (of) <type> ident (integer) semicolon (;) <identifier-list> ident (p) colon (:) <type> ident (Person) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (arr) <component-variable> lbrack (] <index-list> <expression> <simple-expression> <term> <factor> intcon (1) rbrack (] becomes (:=) <expression> <simple-expression> <term> <factor> intcon (10)
---	---

becomes ident (arr) lbrack intcon (2) rbrack semicolon endsy period	semicolon (;) <statement> <assignment-statement> <variable> ident (arr) <component-variable> lbrack (] <index-list> <expression> <simple-expression> <term> <factor> intcon (2) rbrack (] becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (arr) <component-variable> lbrack (] <index-list> <expression> <simple-expression> <term> <factor> intcon (1) rbrack (] <additive-operator> plus (+) <term> <factor> intcon (5) semicolon (;) <statement> <assignment-statement> <variable> ident (p) <component-variable> period (.) ident (age) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (arr)
--	--

	<pre> <component-variable> lbrack ([<index-list> <expression> <simple-expression> <term> <factor> intcon (2) rbrack (] semicolon (;) <statement> <empty> endsy (end) period (.) </pre>
Keterangan	
Pengujian ini digunakan untuk memeriksa kemampuan parser dalam menangani tipe data kompleks, khususnya array dan record. Program memuat deklarasi array, deklarasi record, akses elemen array, dan akses field record. Hasil parse tree menunjukkan bahwa parser mampu membentuk node <array-type>, <record-type>, <selector>, dan <index-list>. Hal ini menunjukkan bahwa parser dapat mengenali struktur tipe kompleks dan akses komponennya sesuai grammar Arion.	

3.24. Pengujian Keduapuluhempat

Input (input24.txt)	Output (parse_tree24.txt)
----------------------------	----------------------------------

```
test > milestone2 > input > ≡ input24.txt
```

```
1  programsy
2  ident (FromTokens)
3  semicolon
4  varsy
5  ident (a)
6  colon
7  ident (integer)
8  semicolon
9  beginsy
10 ident (a)
11 becomes
12 intcon (1)
13 semicolon
14 ident (writeln)
15 lparent
16 ident (a)
17 rparent
18 semicolon
19 endsy
20 period
21 |
```

```
test > milestone2 > output > ≡ parse_tree24.txt
```

```
1  <program>
2    <program-header>
3      programsy
4      ident (FromTokens)
5      semicolon
6    <declaration-part>
7      <var-declaration>
8        varsy
9        <identifier-list>
10         ident (a)
11         colon
12         <type>
13         ident (integer)
14         semicolon
```

```
<program>
<program-header>
  programsy
  ident (FromTokens)
  semicolon
<declaration-part>
  <var-declaration>
    varsy
    <identifier-list>
      ident (a)
    colon
    <type>
      ident (integer)
    semicolon
  <compound-statement>
    beginsy
    <statement-list>
      <statement>
        <assignment-statement>
          <variable>
            ident (a)
          becomes
          <expression>
            <simple-expression>
              <term>
                <factor>
                  intcon (1)
            semicolon
          <statement>
            <procedure/function-call>
              ident (writeln)
              lparent
              <parameter-list>
                <expression>
                  <simple-expression>
                    <term>
                      <factor>
                        <variable>
                          ident (a)
```

	rparent semicolon <statement> <empty> endsy period
Keterangan	
Pengujian ini dirancang untuk membuktikan bahwa parser dapat menerima input berupa file token hasil Milestone 1 melalui mode --from-tokens. Berbeda dengan pengujian lain yang dimulai dari source code, pengujian ini langsung memberikan daftar token ke parser. Hasil parse tree menunjukkan bahwa parser tetap dapat membangun struktur sintaks program tanpa menjalankan lexical analysis ulang. Dengan demikian, implementasi telah memenuhi kebutuhan Milestone 2 bahwa masukan syntax analyzer dapat berupa list token hasil tokenisasi.	

3.25. Pengujian Keduapuluhlima

Input (input25.txt)	Output (error25.txt)
<pre> test > milestone2 > input > ≡ input25.txt 1 program BadWhile; 2 var 3 i: integer; 4 begin 5 i := 0; 6 while i < 3 do i := i + 1; 7 end. 8 </pre>	<pre> test > milestone2 > input > @ error25.txt 1 [ParseError] Body while harus berupa compound-statement: gunakan 'begin ... end' setelah 'do'. Mendapat 'IDENT' 2 </pre> [ParseError] Body while harus berupa compound-statement: gunakan 'begin ... end' setelah 'do'. Mendapat 'IDENT'
Output (token25.txt)	

<pre> test > milestone2 > output > token25.txt 1 programsy 2 ident (BadWhile) 3 semicolon 4 varsy 5 ident (i) 6 colon 7 ident (integer) 8 semicolon 9 beginsy 10 ident (i) 11 becomes 12 intcon (0) 13 semicolon 14 whilesy 15 ident (i) 16 lss 17 intcon (3) 18 dosy 19 ident (i) 20 becomes 21 ident (i) 22 plus 23 intcon (1) 24 semicolon 25 endsy 26 period 27 </pre>	
Keterangan Pengujian ini digunakan untuk memastikan parser menolak struktur while yang tidak memakai compound-statement setelah do. Input berisi bentuk seperti while $x < 5$ do $x := x + 1$;, yaitu body perulangan langsung berupa assignment statement. Hasil pengujian menunjukkan bahwa parser menolak input tersebut dan menghasilkan pesan error sintaks. Hal ini sesuai dengan grammar terbaru yang mewajibkan bentuk valid while $x < 5$ do begin ... end;.	

3.26. Pengujian Keduapuluhenam

Input (input26.txt)	Output (error26.txt)
---------------------	----------------------

```
test > milestone2 > input > ≡ input26.txt
```

```
1  program BadFor;  
2  var  
3    i: integer;  
4  begin  
5    for i := 1 to 3 do i := i + 1;  
6  end.  
7
```

Output (token26.txt)

```
test > milestone2 > output > ≡ token26.txt
```

```
1  programsy  
2  ident (BadFor)  
3  semicolon  
4  varsy  
5  ident (i)  
6  colon  
7  ident (integer)  
8  semicolon  
9  beginsy  
10 forsy  
11 ident (i)  
12 becomes  
13 intcon (1)  
14 tosy  
15 intcon (3)  
16 dosy  
17 ident (i)  
18 becomes  
19 ident (i)  
20 plus  
21 intcon (1)  
22 semicolon  
23 endsy  
24 period  
25
```

programsy
ident (HelloWorld)
semicolon
varsy
ident (a)
comma
ident (b)
colon
ident (integer)
semicolon
beginsy
ident (a)
becomes
intcon (5)
semicolon
ident (b)
becomes
ident (a)
plus
intcon (10)
semicolon

```
test > milestone2 > output > ≡ error26.txt  
1  [ParseError] Body for harus berupa compound-statement: gunakan 'begin ... end' setelah 'do'. Mendapat 'IDENT'  
2
```

[ParseError] Body for harus berupa compound-statement: gunakan 'begin ... end' setelah 'do'. Mendapat 'IDENT'

ident (writeln) lparent string ('Result = ') comma ident (b) rparent semicolon endsy period	
Keterangan	
Pengujian ini digunakan untuk memastikan parser menolak struktur for yang tidak memakai compound-statement setelah do. Input berisi bentuk for i := 1 to 10 do writeln(i);, yaitu body perulangan langsung berupa procedure call. Hasil pengujian menunjukkan bahwa parser menghasilkan syntax error. Dengan demikian, parser berhasil menerapkan aturan bahwa body for harus berupa blok begin ... end, bukan single statement.	

3.27. Pengujian Keduapuluhtujuh

Input (input27.txt)	Output (error27.txt)
<pre>test > milestone2 > input > ≡ input27.txt 1 program ExtraToken; 2 begin 3 end. extra 4</pre>	<pre>test > milestone2 > output > ≡ error27.txt 1 [ParseError] Token tambahan setelah akhir program: 'IDENT' (extra) 2</pre> [ParseError] Token tambahan setelah akhir program: 'IDENT' (extra)
Output (token27.txt)	
<pre>test > milestone2 > output > ≡ token27.txt 1 programsy 2 ident (ExtraToken) 3 semicolon 4 beginsy 5 endsy 6 period 7 ident (extra) 8</pre>	
Keterangan	

Pengujian ini digunakan untuk memastikan parser dapat mendeteksi token tambahan setelah program selesai. Dalam grammar Arion, program yang valid harus berakhir pada token period. Jika masih terdapat token lain setelah akhir program, maka input harus ditolak.

Hasil pengujian menunjukkan bahwa parser menghasilkan syntax error karena terdapat token sisa setelah struktur <program> selesai dibaca. Pengujian ini membuktikan bahwa parser tidak hanya mengecek prefix program yang valid, tetapi juga memastikan seluruh token input telah dikonsumsi sesuai grammar.

3.28. Pengujian Otomatis

Selain pengujian manual melalui screenshot, implementasi juga menyediakan script regression test untuk Milestone 2. Script ini menjalankan seluruh testcase parser, membandingkan output token dan parse tree dengan expected output, serta memastikan testcase invalid menghasilkan error yang sesuai.

Perintah pengujian:

```
make all
./test/milestone2/run_milestone2_tests.sh
```

Hasil pengujian:

Milestone 2 parser regression tests passed.

Pengujian otomatis ini mencakup input valid, input dari file token, serta input invalid seperti while dan for tanpa compound-statement, sehingga membantu memastikan parser tetap sesuai dengan grammar setelah perubahan implementasi.

BAB IV

KESIMPULAN DAN SARAN

4.1. Kesimpulan

Berdasarkan perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa syntax analyzer untuk bahasa Arion telah berhasil dibuat sesuai dengan tujuan Milestone 2. Parser mampu membaca token hasil lexical analysis, memvalidasi urutan token berdasarkan grammar Arion, dan menghasilkan parse tree yang terformat dengan jelas.

Parser diimplementasikan menggunakan metode recursive descent. Setiap nonterminal pada grammar direpresentasikan oleh fungsi parsing tersendiri, sehingga struktur program mudah diikuti dan sesuai dengan aturan produksi yang telah dirancang. Parse tree yang dihasilkan memuat node terminal dan nonterminal yang merepresentasikan struktur sintaks program secara hierarkis.

Implementasi terbaru juga mendukung mode `--parse-only` untuk menguji parser secara mandiri dan mode `--from-tokens` untuk membaca file token hasil Milestone 1. Dengan fitur ini, syntax analyzer dapat diuji baik dari source code maupun langsung dari daftar token. Selain itu, error handling parser sudah mampu memberikan pesan error informatif ketika input tidak sesuai grammar.

Pengujian yang dilakukan mencakup program valid, deklarasi konstanta dan tipe, array, record, procedure, function, case statement, repeat statement, while statement, for statement, input dari token file, serta berbagai kasus invalid. Dengan demikian, parser telah memenuhi kebutuhan utama Milestone 2 sebagai tahap syntax analysis dalam pipeline compiler Arion.

4.2. Saran

Pengembangan berikutnya dapat difokuskan pada peningkatan format visualisasi parse tree agar lebih mudah dibaca pada program berukuran besar. Selain format teks berindentasi, parse tree dapat diekspor ke format lain seperti JSON atau DOT sehingga dapat divisualisasikan sebagai diagram.

Selain itu, error recovery parser juga dapat dikembangkan lebih lanjut. Saat ini parser akan berhenti ketika menemukan syntax error pertama. Pada pengembangan berikutnya, parser dapat dibuat mampu melanjutkan parsing setelah menemukan error tertentu, sehingga dalam satu kali eksekusi program dapat menampilkan lebih dari satu kesalahan sintaks.

LAMPIRAN

Link GitHub: <https://github.com/alyanrrhma/MAR-Tubes-IF2224-2026>

Tabel Pembagian Tugas:

No.	NIM - Nama	Tugas	Persentase
1.	13524017 - Aziza Dharma Putri	1. Implementasi parser 2. Laporan bagian pengujian	25%
2.	13524053 Ahmad Zaky Robbani	1. Revisi perbaikan lexer 2. Laporan bagian kesimpulan dan saran	25%
3.	13524063 - Marcel Luther Sitorus	1. Laporan bagian teori dan rancangan 2. Main.cpp dan Makefile	25%
4.	13524081 - Alya Nur Rahmah	1. Implementasi struktur parse tree 2. Laporan bagian implementasi	25%

DAFTAR PUSTAKA

- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introduction to Automata Theory, Languages, and Computation* (3rd ed.). Pearson Education.
- TutorialsPoint. (n.d.). *Compiler Design - Lexical Analysis*. Accessed online from https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm
- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools*. Pearson Education. Accessed online from [https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles.%20Techniques.%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles.%20Techniques.%20and%20Tools%20(2006).pdf)
- Free Software Foundation. (n.d.). *An Introduction to Makefiles*. Accessed online from https://www.gnu.org/software/make/manual/html_node/Introduction.html
- Theicfire. (n.d.). *Makefile Tutorial by Example*. Accessed online from <https://makefiletutorial.com/>
- Judhi S. (n.d.). *Parse Trees and Conversions*. IF 2124 Teori Bahasa Formal Otomata. Accessed online from https://edunexcontentprodhot.blob.core.windows.net/edunex/2025/81979-Formal-Language-Theory-and-Automata/362559-Module-06/file/1757509600732_slide-08?sv=2024-11-04&spr=https&st=2025-09-10T13%3A06%3A40Z&se=2027-09-10T13%3A06%3A40Z&sr=b&sp=r&sig=BfWy17SqCHFm9Nistwd7qRj0k6k44O53cYS9fUwXqLQ%3D&rsct=application%2Fpdf